k-NN, Naive Bayes and Support Vector Machines — Commentary

Lesson 6 — Technologies for Artificial Intelligence

Fabio Antonini — Università degli Studi dell'Aquila

Not examinable · reading time about 2 hours

Contents

Slide 1 — k-NN, Naive Bayes and Support Vector Machines 5

Part I — Three philosophies, one dataset **5**
Slide 2 — Before we start 5
Slide 3 — Today: the first lesson that offers a choice 5
Question — why are these three separate families, and not variations of one algorithm? 6
Slide 4 — 1,200 pumps, two readings each 6
Question — what does "too far" mean, when the two readings have different units? . 6
Question — why can no model reach 0.96, if the data are simulated? 6
Slide 5 — The shape of the problem: healthy pumps in the middle, faulty all around . . . 7
Question — what is unusual about this picture? 7
Slide 6 — A straight boundary calls all 1,200 pumps faulty 8
Question — the standard deviation is zero: is the model sure of what it found, or does it have no room to improve? 8

Part II — k-nearest neighbours **8**
Slide 7 — k-nearest neighbours: the whole algorithm 8
Worked example — five stored pumps, three values of k 9
Slide 8 — k-NN, drawn: one new pump and its five nearest 9
Question — why does the healthy neighbour not change the answer? 10
Slide 9 — Euclidean distance between two pumps x and x , over their n readings 10
Question — no weight per feature: should the readings be standardised, then? . . . 10
Slide 10 — Scale first, or vibration decides everything 10
Question — does the scaler use the envelope's 3.5 Hz and 0.45 bar? 11
Worked example — who is nearest depends on the units 11
Slide 11 — k sets the balance between bias and variance 11
Slide 12 — Training against cross-validated accuracy, by k 11
Question — why are the training scores measured on 960 pumps, and not on all 1,200? 12
Slide 13 — Accuracy against k, between the baseline and the noise ceiling 13
Slide 14 — The boundary k-NN draws at k = 1, 15 and 401 14
Question — at k = 401, is the model averaging "in the void"? 14
Slide 15 — What k-NN costs 14

Part III — The curse of dimensionality **15**

Slide 16 — An experiment: add columns of pure noise . . . 15
Question — why k = 15 here, and not the best k = 5? . . . 15
Slide 17 — The experiment, drawn . . . 15
Slide 18 — The two real readings are still there, and accuracy still falls . . . 16
Question — why does accuracy fall below the baseline, and not just down to it? . . . 16
Slide 19 — Why: distances stop varying . . . 16
Question — does this hold for any data? . . . 17
Question — why square the distance in the argument? . . . 17
Slide 20 — With n random columns, the nearest point stops being near . . . 17
Worked example — how big a neighbourhood holds 1% of the data . . . 18
Slide 21 — Nearest divided by farthest . . . 18
Slide 22 — In 100 dimensions the nearest point is 70% as far as the farthest . . . 18
Question — is every dataset with 100 columns unsuitable for k-NN? . . . 18
Slide 23 — What the curse is, and is not . . . 19
Question — why did f_classif choose two noise columns, while mutual information found the real ones? . . . 19
Slide 24 — Notebook 1, live . . . 19
Question — which results matter most, if there is no time for every cell? . . . 19
Slide 25 — Break . . . 19

Part IV — Naive Bayes **20**
Slide 26 — Naive Bayes: ask how likely the readings are in each class . . . 20
Slide 27 — Bayes' rule, applied to a class . . . 20
Question — the readings are continuous: shouldn't these be densities? . . . 20
Slide 28 — Drop the denominator: arg max picks the class with the largest numerator . . . 20
Worked example — one pump, two classes, with and without the denominator . . . 21
Slide 29 — The assumption: independent, given the class . . . 21
Question — what does "independent given the class" mean, and why did we assume it for the pumps? . . . 21
Question — an example of features that really are independent given the class . . . 22
Slide 30 — The assumption, written: a product of n densities . . . 22
Worked example — what the assumption saves, counted . . . 23
Slide 31 — Gaussian: each factor is a bell curve . . . 23
Slide 32 — Training computes ten numbers: the widths separate the classes . . . 23
Slide 33 — A pump at 48 Hz, 5.6 bar: there the faulty bell is 86 times higher . . . 24
Question — can the verdict be read here already, without the logarithm slides? . . . 24
Slide 34 — Logs turn the product into a sum, and keep the same winner . . . 25
Question — is underflow "below machine precision"? . . . 25
Slide 35 — 48 Hz, 5.6 bar: scores and their logs . . . 25
Question — where does each log-density come from? . . . 26
Slide 36 — From the two totals to probabilities . . . 26
Question — so the model is 98% sure the pump is faulty: is that the truth? . . . 26
Slide 37 — Software uses only the gap between the logs . . . 26
Question — how does one get from the gap to the formula $1/(1 + e^{(-\text{gap})})$? . . . 27
Slide 38 — A cheap model, built on an assumption that is almost never true . . . 27
Question — what exactly is the assumption on the pumps, with only two readings? . . . 27
Slide 39 — Naive Bayes on the pumps: 0.933 . . . 27
Question — is k-NN better than Naive Bayes, then? . . . 28

Question — can a good accuracy tell me that the assumption was true? 28
Slide 40 — Uncorrelated within each class, but not independent 28
Question — how is the −0.420 computed? 29
Question — and the healthy +0.168? 29
Question — why the square, and not the cube? 29
Question — how would you check the assumption with many features? 29
Slide 41 — The densities are wrong, but the boundary they draw is about right 30
Question — where do 3.17 Hz and 0.41 bar come from? 30
Slide 42 — A second dataset: the signal is an interaction 31
Slide 43 — Each sensor alone: the two classes look the same 31
Question — is the bell the culprit? 31
Question — what do the interacting sensors' numbers show? 32
Worked example — the interaction, counted 32
Slide 44 — 0.404: below the baseline, and below chance 32
Question — how can a model do worse than guessing? 33
Question — then invert every prediction, and it scores 0.596? 33
Question — could a linear model solve it with larger coefficients, or another penalty? 33
Slide 45 — Its probabilities cannot be trusted 33
Question — is this the same distinction as the area under the curve against calibration? 34
Question — is accuracy a kind of ranking too? 34
Slide 46 — Notebook 2, live 34
Question — what do the notebook's conclusions mean? 34

Part V — Support vector machines **35**
Slide 47 — SVM: among the lines that fit, take the widest margin 35
Question — what makes one line different from another, when both have zero training error? 35
Question — why would gradient descent land on one line rather than another? . . . 35
Worked example — distance to a boundary, and the width of the slab 36
Slide 48 — 80 made-up points: three of them fix the line (support vectors) 36
Slide 49 — The strict problem: the widest slab with no point inside 37
Question — what is the cost function here? With linear regression the cost was the MSE, and ridge added a penalty. 37
Slide 50 — Not separable? Charge for the violations 37
Question — can a correctly classified point have non-zero slack? 38
Slide 51 — Three kinds of point: outside the slab, inside it, on the wrong side 38
Slide 52 — The hinge loss stops at zero 39
Worked example — five pumps, two losses 40
Slide 53 — The SVM objective: small w, plus C times the slack 40
Question — is this lesson 3's "loss plus penalty"? 40
Question — so some points are accepted inside the margin, or even on the wrong side? 40
Slide 54 — C is the price of a training error 41
Question — what happens at the two ends? 41
Question — in what sense is the margin "deformed"? Is it not always straight? . . . 41
Worked example — two candidate boundaries, three prices 41
Slide 55 — The best straight line is still a straight line 41
Slide 56 — On the pumps: a straight line leaves almost every pump in its slab 42
Question — what exactly are the circled points? 42

Slide 57 — Many support vectors: a free warning . . . 43
Question — does a high share prove the kernel is wrong? . . . 43
Slide 58 — A new coordinate makes the classes separable . . . 43
Slide 59 — The same pumps, lifted . . . 43
Slide 60 — The SVM's solution: a weighted sum of pumps . . . 44
Question — where does $w = \sum_i a_i y_i x_i$ come from? It looks dropped from above. . . . 44
Question — what is the dual formulation, intuitively? . . . 44
Question — were the 922 zero-weight pumps ignored during training? . . . 44
Slide 61 — So the new coordinates are never computed . . . 45
Slide 62 — The kernel trick by hand: 25 = 25 . . . 45
Question — why is it always the same number, and why the √2? . . . 45
Question — how do I find $K(x_i, x_j)$ without knowing $\phi(x_i) \cdot \phi(x_j)$? . . . 45
Slide 63 — The RBF kernel: similarity falls with distance . . . 46
Question — does a similarity of 0.61 mean a 61% chance that two pumps share a class? 46
Question — is = 0.5 scikit-learn's fixed default? . . . 46
Worked example — how similar two pumps are, by distance . . . 47
Slide 64 — Training: one weight per pump . . . 47
Question — who are x_i and x, during training and during prediction? . . . 47
Slide 65 — The SVM's score at 48 Hz, 5.6 bar: Σ a y K(x , x) + b = +1.883 . . . 47
Question — how should the total be read? . . . 48
Slide 66 — One pump's vote, drawn . . . 49
Slide 67 — Training against cross-validated, three settings . . . 49
Question — is it or C that overfits? . . . 50
Slide 68 — How far one support vector's vote reaches, for four values of . . . 50
Slide 69 — Overfitting you can see: the boundary at the three settings . . . 51

Part VI — Closing **51**
Slide 70 — Notebook 3, live . . . 51
Slide 71 — The three families compared, on the same pumps and the same folds . . . 51
Question — so the RBF SVM is the best model of the three? . . . 52
Slide 72 — How to choose, in practice . . . 52
Question — "very many features, little data: Naive Bayes", after an hour of its failures? 53
Question — and when the decision has to be explained? . . . 53
Slide 73 — What to take away . . . 53
Slide 74 — Homework . . . 54
Question — what does the last part of the exercise ask about the ceiling? . . . 54

What this document is. A companion to the slides of lesson 6: for every slide, what it means, the steps it leaves implicit, and the questions it tends to raise, answered under the slide they belong to. It is a second explanation of the same material, in a different voice from the handout, which remains the reference for the derivations. It is **not examinable**: nothing here is needed for the exam that is not already in the handout and the notebooks.

Slide numbers and titles are the ones in the lesson's slide deck, in `Slides/` (74 slides). Where a section mentions "the notes", it means the speaker notes stored in the slide deck, which PowerPoint shows below each slide. The questions were asked while the lesson was being studied, or are the ones an independent review expected students to ask.

Worked examples sit under ten slides (7, 10, 20, 28, 30, 43, 47, 52, 54, 63): a handful of invented values each, small enough to work through by hand. Their numbers are not the lesson's data; every one is computed by `commentary_examples.py`, beside this file, which also writes them here.

Slide 1 — k-NN, Naive Bayes and Support Vector Machines

The title slide, but it announces the idea that runs through the whole lesson: these are not just three more classifiers, they are three different ways of thinking about what a model is.

- **k-nearest neighbours (k-NN)** reasons from examples. It keeps the training data and, when a new point arrives, looks at what happens near it.
- **Naive Bayes** does almost the opposite. It compresses the data into a small probabilistic model, at the price of a strong assumption: the features are independent once the class is known.
- **The support vector machine (SVM)** reasons geometrically. It looks for the boundary that leaves the **widest margin** around it and, when a straight boundary is not enough, changes the space the data lives in, through a kernel.

In one line: **k-NN remembers, Naive Bayes assumes, the SVM separates.**

Part I — Three philosophies, one dataset

Slide 2 — Before we start

The slide links lesson 6 to lesson 5. Lesson 5 built the tools for judging whether a result can be believed: cross-validation, baselines, leakage, the comparison between training and validation scores, bias and variance. Changing model does not retire them. Lesson 6 is where **model selection** begins in earnest: knowing how to train a model is not enough; you have to decide which *family* of models suits the structure of the problem. The opening discussion of Exercise 5 is about a result that looked too good: a colleague's notebook whose score was wrong in more than one way, and the honest number that replaces it. Exercise 5 was set when lesson 5 finished, on 13 November; exercise 4 had been discussed at the start of that same day.

Slide 3 — Today: the first lesson that offers a choice

Until now the course has followed one path: data, preprocessing, a linear model, optimisation, evaluation. Now a new question appears: **which family of models should I choose?** The three answers of the day are:

- **k-NN — remember and vote.** It assumes no shape for the boundary and fits nothing; it keeps the examples.
- **Naive Bayes — make a strong assumption.** It assumes the features are independent given the class, and gets in return a model that is very easy to estimate.
- **SVM — margins and kernels.** It looks for a robust separation, and can change the feature space without ever writing the new features down.

The distinction matters more than the small differences between their accuracies, and it is one of the lesson's closing messages: **choosing the right family can count far more than fine-tuning the wrong one.**

Question — why are these three separate families, and not variations of one algorithm?

Because each one rests on a different idea of what a classifier is. k-NN classifies by the neighbours of the new point; Naive Bayes models how the readings are spread inside each class and turns that into a probability with Bayes' rule; the SVM chooses a boundary by its margin and can change coordinates with a kernel. Different mechanisms make different assumptions, and the assumptions are exactly what decides which family suits a problem. Even k-NN has one: that points near each other, in the distance you chose, tend to share a label. Slides 16 to 23 show what happens when that stops being true.

Slide 4 — 1,200 pumps, two readings each

The dataset for most of the lesson: **1,200 simulated industrial pumps**, two readings each, vibration in hertz (Hz) and pressure in bar. A pump is healthy when it runs close to its design point, 42 Hz and 5.6 bar, and faulty when it drifts too far from it **in any direction**. So the rule is not of the kind "faulty if the vibration is above a threshold": a vibration can be a problem because it is too high *or* too low, and the same holds for the pressure.

Question — what does "too far" mean, when the two readings have different units?

Each reading is measured in its own steps, 3.5 Hz for vibration and 0.45 bar for pressure, and the two are combined into one distance:

$$r = \sqrt{\left(\frac{x_{\text{vib}} - 42}{3.5}\right)^2 + \left(\frac{x_{\text{press}} - 5.6}{0.45}\right)^2}.$$

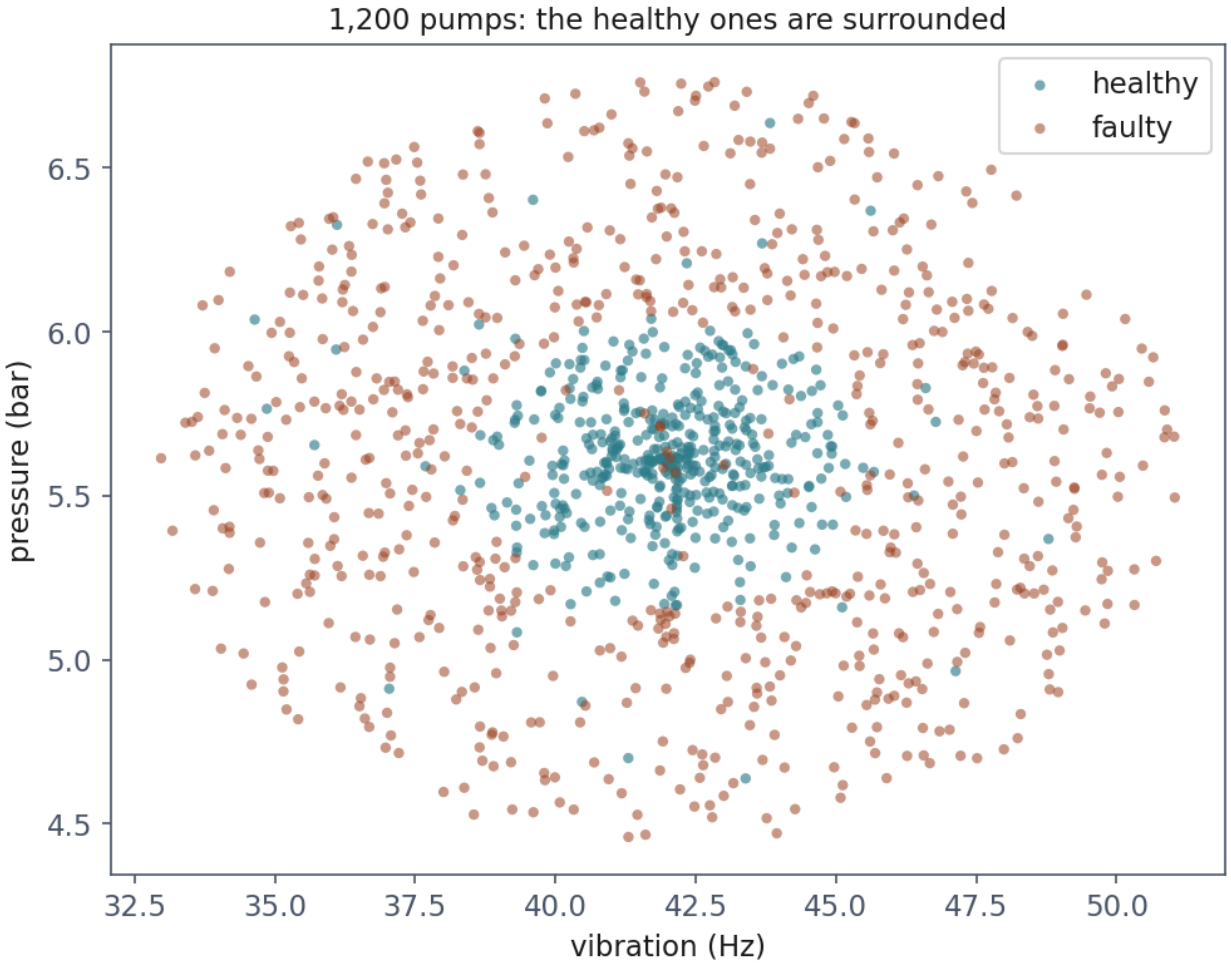
The pump is faulty when $r > 1$. The boundary $r = 1$ is an ellipse around the design point, 3.5 Hz wide on each side and 0.45 bar high, not two separate ranges: a pump with each reading inside its own range can still be faulty, in the corners of the box. This is the picture to draw on the board before the next slide.

Question — why can no model reach 0.96, if the data are simulated?

The 4% of flipped labels are put there on purpose. They are **errors in the reference labels**, the way a real inspection sometimes labels a pump wrongly; the readings themselves are untouched. Being simulated is what lets us know the rate exactly. A pump whose label was flipped will be called wrongly by any model that follows the true rule, so on new pumps about 0.96 is the best accuracy to expect: the noise ceiling. On this particular sample 49 labels were flipped, so the rule itself scores 1,151 of 1,200, 0.959. A finite validation set can land a little either side of 0.96; a score well above it calls for a search for leakage before anything else.

About **61.3%** of the pumps are faulty, 735 of 1,200, so a classifier that always says "faulty" already scores 0.613. That baseline is the yardstick for every result that follows.

Slide 5 — The shape of the problem: healthy pumps in the middle, faulty all around



1,200 pumps. The healthy ones are surrounded, because a pump can fail by running too slow as well as too fast. No straight line separates these classes.

In the plane of vibration and pressure, the healthy pumps occupy a central region and the faulty ones surround it. In units of 3.5 Hz and 0.45 bar the healthy region is a **disc**; on the Hz and bar axes of the figure it is an **ellipse**, because the two steps differ.

A linear boundary in two dimensions, $w_1 x_1 + w_2 x_2 + b = 0$, is a **straight line**, and a line splits the plane into two half-planes. What this problem needs is "inside is healthy, outside is faulty", in every direction, and no single line can enclose a central region. So the problem is not "find a better line": **no line can represent this structure at all.** That anticipates an idea that returns with the SVM: if the family of models (the *hypothesis class*) is wrong, a better optimiser cannot help.

Question — what is unusual about this picture?

The usual textbook picture of classification is two clouds of points side by side, one class on the left and one on the right, with a line between them. Here one class surrounds the other. That shape was not chosen to make the problem hard: it follows from how a pump fails, by running too low as well as too high. Every problem of the form "inside the tolerance is good, outside is bad" has it —

a machined shaft whose diameter must be 20 mm plus or minus 0.1 mm, a supply voltage, a body temperature — and pass/fail inspection of this kind is a large share of classification in industry.

Slide 6 — A straight boundary calls all 1,200 pumps faulty

Model	Cross-validated accuracy, mean ± standard deviation over 5 folds
Majority baseline	0.613
Logistic regression	0.613 ± 0.000

Logistic regression is not doing "badly": it scores **exactly the baseline**, because it predicts "faulty" for every one of the 1,200 pumps. On these pumps, with a straight boundary, that is genuinely the best answer available; it is a property of this data, not a law about linear models.

Question — the standard deviation is zero: is the model sure of what it found, or does it have no room to improve?

Neither. The standard deviation is exactly zero because the folds are **stratified**: each holds 240 pumps, of which 147 are faulty, and a model that calls every pump faulty scores 147 / 240 = 0.6125 on every one of the five. Nor is the model confident: its probability of "faulty" runs only from 0.579 to 0.641 across the 1,200 pumps, close to the base rate for every pump. It does not tell pumps apart at all. Room for improvement there is, plenty of it, but not within straight lines: the rest of the lesson changes family. The diagnostic to keep: **a score that does not move across folds often means a model that is not using its input.**

Part II — k-nearest neighbours

Slide 7 — k-nearest neighbours: the whole algorithm

The whole of k-NN fits in one sentence: to classify a new point, find the k stored examples closest to it and take a majority vote. With $k = 5$ and neighbours labelled H, H, F, H, F, there are three H against two F, so the prediction is healthy. With two classes an odd k avoids ties.

There is **no real training**. Logistic regression learned coefficients $w_1, \ldots, w_n, b$; the `fit()` of k-NN stores the training set, and with a scaler in the pipeline it also computes each column's mean and standard deviation, the only numbers estimated. That is why it is called a **lazy learner**, and the work it does not do at `fit` time is paid for at prediction time. In scikit-learn the whole algorithm is one class, `KNeighborsClassifier(n_neighbors=5)`; 5 is also its default.

The one sentence hides two decisions: what "closest" means (slide 9) and how many neighbours vote (slides 11 to 13). Both change the answer.

Worked example — five stored pumps, three values of k

Distances from a new pump to the five stored ones (standardised units):

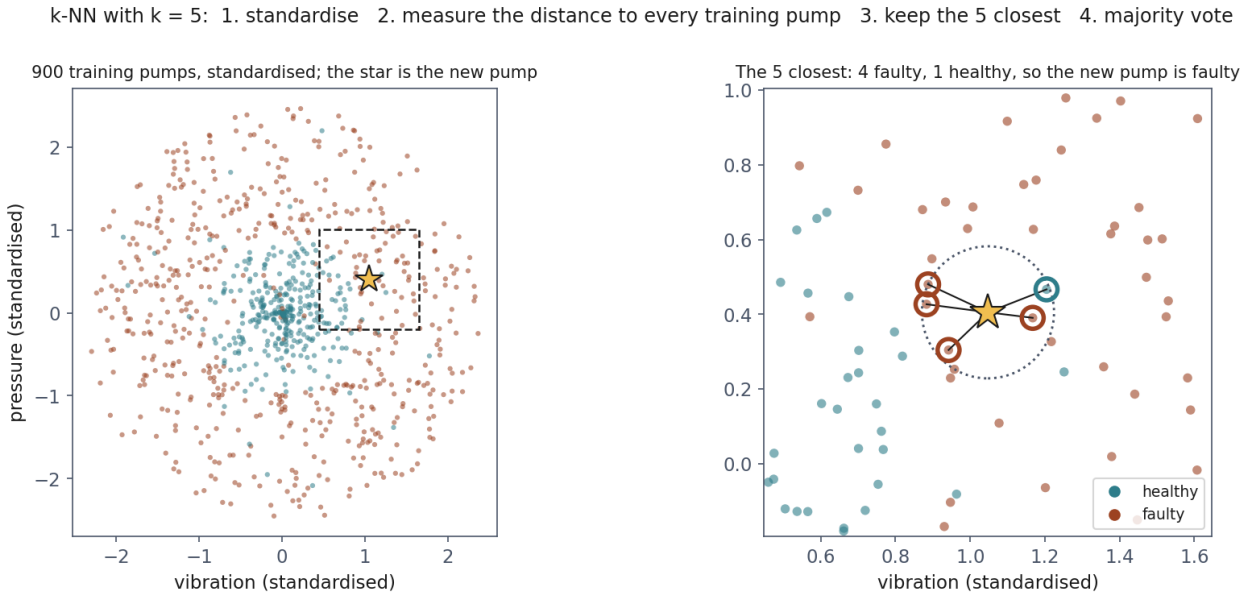
pump	distance	label
A	0.4	faulty
B	0.7	healthy
C	0.9	healthy
D	1.3	faulty
E	1.6	faulty

- $k = 1$: the nearest, A (faulty) → **faulty**
- $k = 3$: the 3 nearest, A (faulty), B (healthy), C (healthy) → **healthy**
- $k = 5$: the 5 nearest, A (faulty), B (healthy), C (healthy), D (faulty), E (faulty) → **faulty**

Nothing was fitted and nothing is estimated: the answer is a vote, and k decides who gets one. The same pump changes class twice as k grows — which is why k is chosen by cross-validation and not by taste.

Slide 8 — k-NN, drawn: one new pump and its five nearest

The previous slide stated the whole algorithm in one sentence; this one does it once, on the pumps, so the four steps can be followed with the eye. The data are notebook 1's split: 900 stored pumps, the other 300 held out, both readings standardised by a scaler fitted on the 900 alone. The new pump sits at 46 Hz and 5.8 bar, the gold star. Rust is faulty and teal is healthy, here and on every figure.



Left, every stored pump and the box the right panel enlarges. Right, the five closest pumps, circled in their class colour; the dotted circle reaches the fifth.

Four of the five neighbours are faulty and one is healthy, so the vote is four to one and the new pump is classified faulty. The distances are measured in standardised units, which is why the circle is round.

Question — why does the healthy neighbour not change the answer?

Because it is outvoted, and because it is wrong. Notebook 1 prints each neighbour's label next to what the rule that generated the data says: the healthy one is pump 1058, at 46.60 Hz and 5.83 bar, well outside the envelope. By the rule it is faulty; its label is one of the 4% that were flipped. The star itself is faulty by the rule too. With $k = 1$ the verdict would rest on the single closest pump, and if that pump carried a flipped label the verdict would be wrong. A vote over five neighbours lets one odd label be outvoted, which is the first hint of why k is a trade-off.

Slide 9 — Euclidean distance between two pumps x and x , over their n readings

The Euclidean distance between two pumps x and x', here the new pump and one stored pump, over their n readings,

$$d(x, x') = \sqrt{\sum_{j=1}^{n} (x_j - x'_j)^2} :$$

for each reading take the difference, square it, add the squares over the n readings, take the square root. scikit-learn's default, `metric="minkowski"` with `p=2`, is exactly this.

The formula matters more than it looks. k-NN has no coefficients with which to say "vibration matters, pressure a little less, feature 17 not at all": every feature enters the distance with the same weight, in its own units. A linear model can learn $w_1 = 4$ and $w_2 = 0.001$ and so learn that x_2 hardly counts; k-NN has no such mechanism. Hence the notes' phrase: **the distance function is the model.**

Question — no weight per feature: should the readings be standardised, then?

Yes, and that is the next slide. Each column is shifted by its mean and divided by its standard deviation, both computed on the training part only, so that a difference of 1 means "one typical spread" for either reading. Standardising does not find the right weights, though: it gives every column the *same* weight, so a useless column counts as much as a useful one. Slides 16 to 23 show what that costs.

Slide 10 — Scale first, or vibration decides everything

Vibration runs from 33 to 51 Hz across the 1,200 pumps, a range of 18 Hz; pressure from 4.5 to 6.8 bar, a range of 2.3 bar (notebook 1 prints both). Suppose pump A differs from a query by 10 Hz in vibration and nothing in pressure, so $d_A = 10$, and pump B has the same vibration and differs by 1 bar, so $d_B = 1$. k-NN concludes that B is ten times closer, but only because of the units: measure pressure in millibar and the same difference becomes 1,000, and suddenly pressure dominates.

The fix is to standardise, $z_j = (x_j - \text{mean}_j)/\text{sd}_j$, so that a difference of 1 means one standard deviation on every feature. And the scaler goes **inside the pipeline**,

`make_pipeline(StandardScaler(), KNeighborsClassifier(5))`, so that during cross-validation the means and standard deviations are computed on the training rows of each fold only. Otherwise the scaling leaks.

Question — does the scaler use the envelope's 3.5 Hz and 0.45 bar?

No, and the two are easy to confuse. The steps 3.5 Hz and 0.45 bar belong to the rule that generated the labels; the scaler knows nothing about it. It estimates the data's own spread, which on all 1,200 pumps is 3.78 Hz and 0.471 bar, and inside cross-validation it estimates it again on each fold's training rows.

Worked example — who is nearest depends on the units

A new pump at 42 Hz, 5.6 bar. Two stored pumps: A at 45 Hz, 5.6 bar; B at 42.5 Hz, 6.3 bar. Suppose vibration typically spreads by about 6 Hz and pressure by about 0.3 bar.

distance to	A	B	nearest
raw units	3.00	0.86	**B**
standardised	0.50	2.33	**A**

Raw, A is $\sqrt{3^2+0^2}$ away and B $\sqrt{0.5^2+0.7^2}$; standardised, A is $\sqrt{(3/6)^2+0^2}$ and B $\sqrt{(0.5/6)^2+(0.7/0.3)^2}$.

In raw units 0.7 bar looks smaller than 3 Hz, so B wins — although 0.7 bar is more than two typical spreads of pressure and 3 Hz is half a spread of vibration. Standardised, each reading is measured against its own spread and A is the real neighbour.

Slide 11 — k sets the balance between bias and variance

k is the most intuitive example of lesson 5's bias-variance trade-off.

Small k: the vote follows each stored pump, those with flipped labels included: low bias, high variance. "High variance" has a concrete meaning: draw another set of pumps from the same machines and the prediction at a given point changes a lot, because it rests on one or two pumps, and the flipped labels are what makes it move. In the extreme, $k = 1$: every stored pump is its own nearest neighbour, at distance zero, so it votes for its own label and the training accuracy is exactly 1 — provided no two stored pumps have identical readings with different labels, which with continuous readings does not happen. That does not mean the model generalises; it means it **memorises**.

Large k: the vote reaches pumps across the edge of the envelope, and near the edge the larger class wins. Variance falls and bias rises. With k equal to every stored pump, every pump gets the majority class: the 0.613 baseline.

Slide 12 — Training against cross-validated accuracy, by k

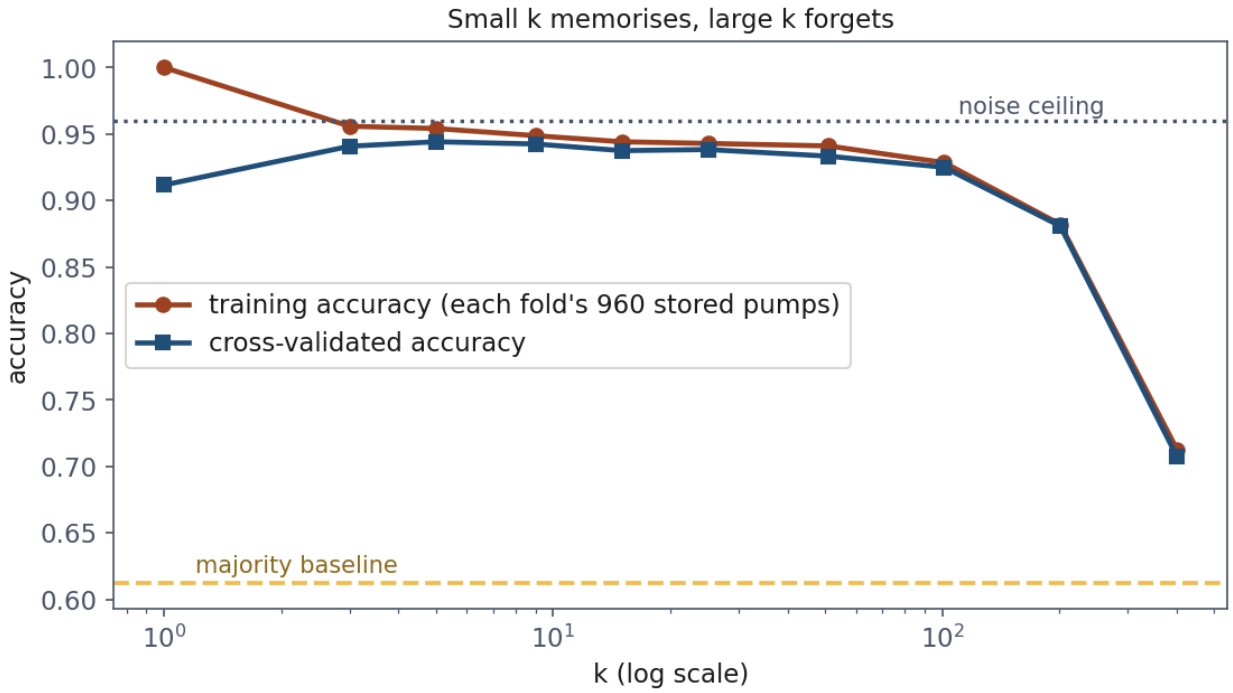
k	Training (each fold's 960 stored pumps)	Cross-validated
1	1.000	0.912
5	0.954	**0.944**
15	0.944	0.938
51	0.941	0.933
401	0.713	0.708

At $k = 1$, 1.000 against 0.912 is high variance: the model memorises the 4% of labels that were deliberately flipped. At $k = 5$ the neighbourhood is large enough to outvote many noisy points and still small enough to follow the real shape: 0.944. At $k = 401$, training 0.713 and validation 0.708: this is not overfitting, because **the training score is low too**. It is the other side of the trade-off, underfitting, or high bias. Between 5, 15 and 51 the change is gradual: the choice of k is forgiving over a wide range.

Question — why are the training scores measured on 960 pumps, and not on all 1,200?

So that both columns come from the same models. In each of the five rounds the scaler and k-NN are fitted on 960 pumps and score the 240 held out; the training score is measured in the same round, on the 960 the model stores, and both are averaged over the five rounds (`cross_validate` with `return_train_score=True`). Scoring one model fitted on all 1,200 against models fitted on 960 would compare different neighbourhoods: 401 of 1,200 pumps is a smaller share than 401 of 960, and at large k the gap between the columns would be mostly an artefact. Each stored pump is among its own k nearest, which is why the training score is optimistic for every k, not only for $k = 1$.

Slide 13 — Accuracy against k, between the baseline and the noise ceiling

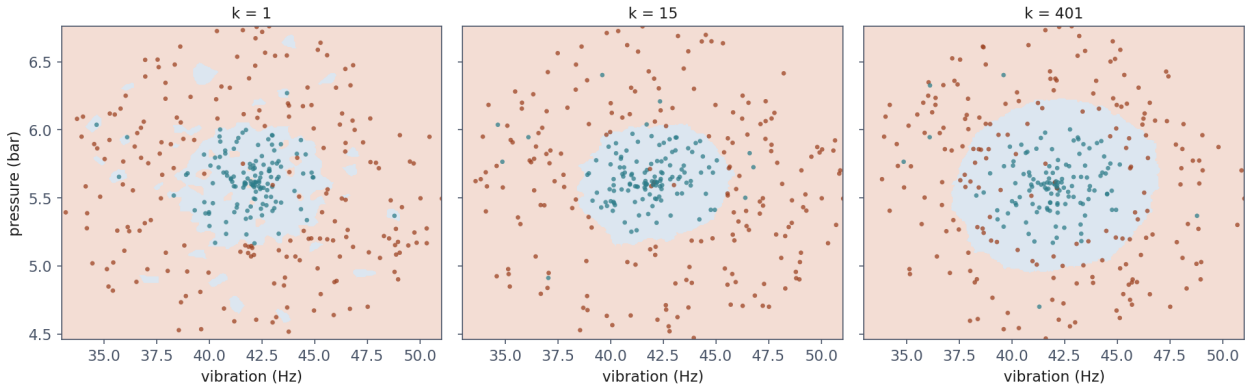


The two curves and the two lines that bound them: the noise ceiling at 0.96 and the majority baseline at 0.613. Note where the training curve starts.

The k axis is logarithmic: 1, 3, 5, 9, 15, 25, 51, 101, 201, 401. The training accuracy starts at 1.0 at $k = 1$ — above the 0.96 ceiling, which only memorising the flipped labels can achieve — and falls as k grows. The validation accuracy starts lower, rises, peaks and falls again: the classic bias-variance shape, its maximum around $k = 5$. Where the two curves meet and then fall together, the variance is gone and bias has taken over.

One subtlety from the notes: the gap between training and validation is **not, in mathematical terms, "the variance"**. A large gap is a symptom consistent with high variance, while training and validation scores that are both low and close together are typical of high bias.

Slide 14 — The boundary k-NN draws at k = 1, 15 and 401



The same data at three values of k. At k = 1 the boundary is ragged, with islands around individual pumps with flipped labels. At k = 15 it is a clean ellipse, close to the envelope that generated the data. At k = 401 the healthy region has ***inflated*** *past the true envelope and swallows faulty pumps.*

At $k = 1$ the boundary is jagged, with islands both ways: teal patches in the faulty ring and rust holes in the healthy region, one per flipped label. That is variance made visible: the model has carved out a small territory for each one. At $k = 15$ the central region is a clean ellipse, close to the true envelope, 42 ± 3.5 Hz by 5.6 ± 0.45 bar. At $k = 401$ the healthy region has grown.

Question — at k = 401, is the model averaging "in the void"?

Not quite: the region grows for a specific reason. Healthy pumps are packed tightly in the middle, while faulty ones are spread thinly over the wide ring — 465 healthy against 735 faulty, on a much larger area. A pump just outside the envelope finds among its 401 nearest most of the healthy pumps, so "healthy" wins there too: the healthy region inflates, and faulty pumps near the edge are called healthy. The model has not collapsed to one class (it still scores 0.708); it has stopped following the boundary and started averaging across it.

Slide 15 — What k-NN costs

So far k-NN looks almost too good: storing the pumps costs nothing. But the cost has only moved to prediction. With m stored pumps and n features, a direct query computes the distance to all m of them, about $m \times n$ operations per new pump, plus picking the k smallest. With 1,200 pumps that is nothing; with ten million rows and a thousand queries a second it is the whole engineering problem. scikit-learn avoids comparing with every row by default (a k-d tree or a ball tree, chosen by `algorithm="auto"`), which helps with few features and stops helping with many; at millions of rows, approximate nearest-neighbour libraries take over (this lesson's Resources document).

There is a second cost: **the dataset is the model.** To deploy k-NN you deploy the stored examples, with consequences for memory, latency, privacy and possibly the law: if they are medical records, shipping the model ships the records (lesson 2's Resources document, on the GDPR). The flip side: erasing one person from a k-NN model is immediate — delete the row — where a fitted model would need retraining. A deployed logistic regression, by contrast, is a vector w and an intercept b.

Part III — The curse of dimensionality

Slide 16 — An experiment: add columns of pure noise

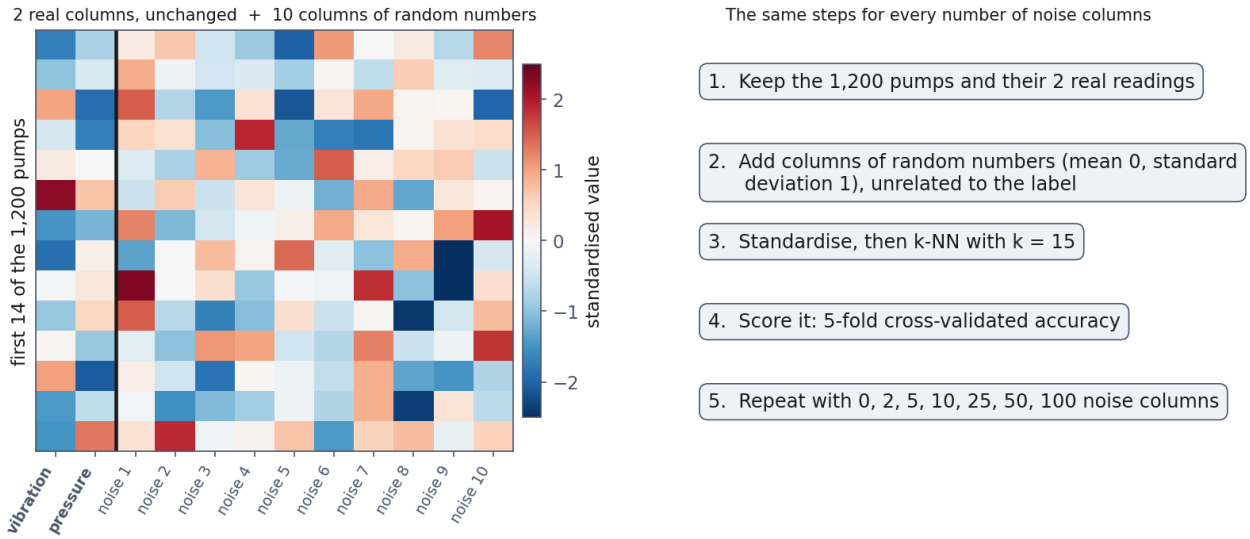
The experiment: keep the two informative readings, vibration and pressure, and append columns of pure noise, numbers drawn from a normal distribution with mean 0 and standard deviation 1. `load_with_noise()` leaves the two original columns **exactly as they were** and only adds independent columns. So **no useful information is removed**: the problem is still solvable as well as before, up to the 0.96 ceiling. After standardising, each noise column weighs in the distance exactly as much as each real reading — the "same weight to every column" of slide 10.

Question — why k = 15 here, and not the best k = 5?

On the two real columns $k = 15$ scores 0.938 against 0.944, almost the same, and a vote over 15 pumps is less thrown by one bad neighbour than a vote over 5. So if accuracy falls as noise columns are added, the noise columns did it, not a small k.

Slide 17 — The experiment, drawn

The previous slide described the experiment in words; this one draws it, before the result. Nothing in it is a finding yet: it shows what k-NN is given, and what is repeated.



Left, the table k-NN receives with ten noise columns: the first 14 of the 1,200 pumps, every column standardised, colour for the value. Right, the five steps, repeated for every number of noise columns.

The two columns left of the black line are vibration and pressure, the same in every run. Everything to the right is random. The thing to see: **after standardising, the two real columns look exactly like the noise ones** — the same colours, the same range. Nothing in this table tells k-NN which columns matter. The model is the same each time, standardised k-NN with $k = 15$, scored by 5-fold cross-validation; only the number of random columns changes.

Slide 18 — The two real readings are still there, and accuracy still falls

Total columns	Accuracy	Accuracy − baseline 0.613
2	0.938	+0.325
12	0.762	+0.149
27	0.662	+0.049
52	0.602	−0.011
102	0.578	−0.035

With two features, 0.938. Ten useless columns bring it to 0.762; fifty, to 0.602, **below the majority baseline** of 0.613, although vibration and pressure are still there unchanged. The signal was not lost: **the geometry k-NN used to find the signal was destroyed.** With 2 real columns out of 102, the noise columns make up almost all of each sum of squares, so the 15 "nearest" pumps are nearest in the noise.

Question — why does accuracy fall below the baseline, and not just down to it?

Because a vote among nearly random pumps is worse than always saying "faulty". The 15 "nearest" pumps are by now close to 15 pumps taken at random. A random pump is faulty with probability $735/1{,}200 = 0.6125$, so the vote says "faulty" when at least 8 of the 15 are faulty, which happens with probability 0.815. The vote ignores the pump being classified, so it is right when it says "faulty" to a faulty pump or "healthy" to a healthy one: $0.6125 \times 0.815 + 0.3875 \times 0.185 = 0.571$, against 0.578 measured at 102 columns. The vote still says "healthy" 18.5% of the time for no reason, and those answers are right only as often as healthy pumps occur; always saying "faulty" never makes that mistake (handout section 3.1, Checks).

Slide 19 — Why: distances stop varying

This is **a separate experiment**, with no pumps, no labels and no model: 1,000 random points with n columns, each a uniform number between 0 and 1 (a square for $n = 2$, an ordinary cube for $n = 3$). For a new point, measure the distance to its nearest and to its farthest neighbour. It measures *why* the pumps' accuracy fell.

The intuition that sticks: to be near in 100 dimensions a point must be near in all 100 coordinates **at once**, and almost no point is. Every pair is near in some coordinates and far in others, and all pairs end up equally mixed. A second picture: one die thrown gives anything from 1 to 6, but the sum of 100 dice is always close to 350 — and every squared distance is such a sum, one term per coordinate.

The arithmetic behind it: write $d^2 = \sum_{j=1}^{n} (x_j - x'_j)^2$. With independent, similarly distributed coordinates each term has the same mean e and variance v, so

$$\mathbb{E}[d^2] = ne, \qquad \mathrm{SD}(d^2) = \sqrt{nv}, \qquad \frac{\mathrm{SD}(d^2)}{\mathbb{E}[d^2]} = \frac{1}{\sqrt{n}} \frac{\sqrt{v}}{e}.$$

The centre grows like n, the spread only like $\sqrt{n}$, so relative to their size the distances bunch up: for the uniform cube the relative spread of d^2 is 0.84 in two dimensions and 0.12 in one hundred. Distances do not go to zero; they grow. They become **alike relative to their size**.

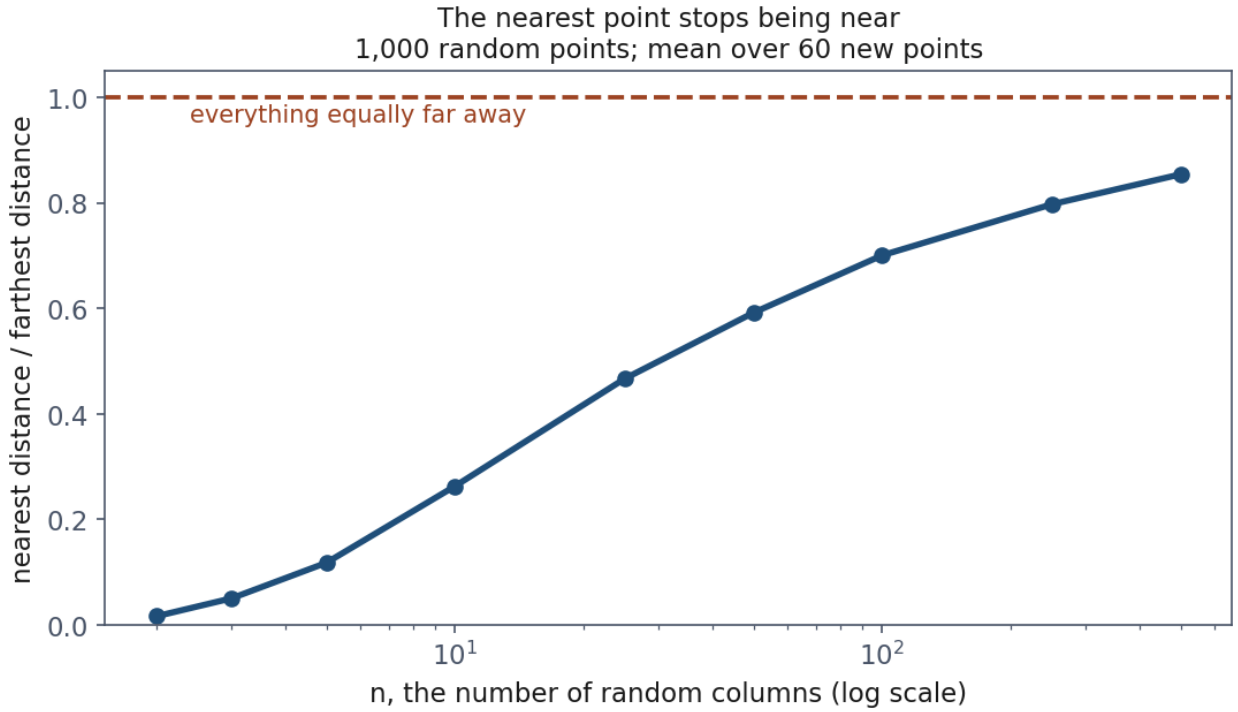
Question — does this hold for any data?

It holds whenever the columns behave like independent random numbers of comparable scale — which is exactly what the noise columns of the pump experiment are. Real data with a few strongly informative columns can keep meaningful neighbourhoods in many dimensions; what destroys them is adding columns that carry nothing. That is why the lesson's remedy (slide 23) is to choose the columns, not to give up on distances.

Question — why square the distance in the argument?

Because d^2 is a plain sum of n independent terms, and for sums the means add and the variances add, which makes the arithmetic one line. The square root at the end does not change the conclusion: at $n = 100$ the relative spread of the distance itself is 0.059, half that of its square.

Slide 20 — With n random columns, the nearest point stops being near



As the number of random columns grows, the nearest point stops being near. Each point is the mean over 60 new points, each measured against the same 1,000.

The slide turns the argument into a quantity you can picture: the distance to the nearest point divided by the distance to the farthest. With 2 columns the nearest is about 60 times nearer than the farthest, so "the nearest neighbour" means something strong. With 100 columns the nearest is 70% as far as the farthest: it still exists, but it is not much closer than anything else. The ratio approaches 1 without reaching it, and most of the rise is over by 50 columns. This is the same effect that hit the pumps, measured on random points.

Worked example — how big a neighbourhood holds 1% of the data

Points spread evenly in a cube of side 1. A smaller cube that contains 1% of them must have volume 0.01, so its side is $0.01^{1/d}$:

dimensions d	side of the 1% cube	as a fraction of each axis
1	$0.01^{1/1}$	**0.010**
2	$0.01^{1/2}$	**0.100**
10	$0.01^{1/10}$	**0.631**
100	$0.01^{1/100}$	**0.955**

In two dimensions the 1% neighbourhood spans a tenth of each axis — local. In ten it spans 63% of every axis, and in a hundred nearly all of it. A "nearest neighbour" found that way is not near in any useful sense; it is just the least far.

Slide 21 — Nearest divided by farthest

n, random columns	nearest ÷ farthest, mean over 60 new points
2	0.016
10	0.263
50	0.592
100	0.701
500	0.855

In two dimensions the nearest point is very close compared with the farthest, 0.016. With ten columns, an ordinary-looking table, the nearest is already a quarter of the way to an arbitrary point; with fifty, more than half; with a hundred, 70%.

Slide 22 — In 100 dimensions the nearest point is 70% as far as the farthest

k-NN rests on an implicit assumption: the points nearest the query are especially relevant to it. If the nearest point is about as far as a random one, then back on the pumps a vote among the 15 nearest is almost a vote among 15 pumps taken at random — and slide 18 showed that such a vote scores about 0.571, **below** the 0.613 base rate, not at it. That is why the 52- and 102-column rows fell below the baseline.

Question — is every dataset with 100 columns unsuitable for k-NN?

No. The experiment used independent random columns, and the pumps failed because columns carrying nothing were added. A dataset whose 100 columns all carry signal, or whose useful information lives in a few of them, behaves differently. The practical lesson is the next slide's: choose the columns, inside the pipeline, before any distance is computed.

Slide 23 — What the curse is, and is not

The curse of dimensionality does **not** mean "machine learning does not work in high dimension": many models handle thousands or millions of features. A linear model can switch off a useless feature with a coefficient near zero — provided it has many more examples than columns, as here, 1,200 against 102; with few examples even a linear model overfits noise columns, which is why lesson 3 regularised. What suffers is methods that rely directly on distances: k-NN, k-means, the radial basis function kernel. In k-NN a feature stays in $\sum_j (x_j - x'_j)^2$, and no coefficient removes it. Hence the value of **choosing the columns**, which, as lesson 5 insisted, must happen **inside the cross-validation pipeline**, so that each fold chooses on its own training part only.

Question — why did f_classif choose two noise columns, while mutual information found the real ones?

Measured in notebook 1 on 52 columns (2 real, 50 noise), $k = 15$, the same folds: with no selection 0.602; `SelectKBest(f_classif, k=2)` keeps two noise columns and scores 0.562, worse than doing nothing; `SelectKBest(mutual_info_classif, k=2)` keeps vibration and pressure and gets back 0.938. `f_classif` scores a column by how far apart the class means are, and on the ring healthy and faulty pumps have almost the same mean vibration and pressure (slide 32 shows them), so it finds nothing in the real columns and ranks two noise columns first by chance. `mutual_info_classif` detects any dependence between a column and the label, not only a shift of the mean.

Slide 24 — Notebook 1, live

The slide adds no theory; it is where the theory is checked by experiment. Its table pairs each section of the notebook with the slides it comes from and the scikit-learn calls it uses, so that a cell you are stuck on leads you back to the idea. The notebook shows, in order, the straight line failing, k-NN written out in eight lines and checked against `KNeighborsClassifier` (the two agree on all 300 held-out pumps), the effect of different values of k, the noise columns, the geometry degrading with the dimension, and the choice of columns. The last row names the alternative: atoms 1 to 5 of `Consolidation/06_lesson_6_atoms.ipynb`, the same ideas on small predict-then-run cells.

Question — which results matter most, if there is no time for every cell?

First the strong k-NN score on the two real readings, then the fall below the baseline with fifty noise columns, then the column selection inside the pipeline getting it back: together they show that the signal never disappeared, the distance was made uninformative.

Slide 25 — Break

The break also marks a sharp turn. k-NN keeps practically the whole dataset; the next model does almost the opposite, and compresses the training set into a handful of numbers.

Part IV — Naive Bayes

Slide 26 — Naive Bayes: ask how likely the readings are in each class

What we want is $P(y = c \mid x)$: the probability of the class given the readings of this pump. The data does not hand us that quantity directly. Bayes' rule **turns the question around**: instead of "given these readings, which class?" it asks "given the class, how likely are these readings?", which the data can answer — take the faulty pumps alone and look at where their readings fall, then the healthy ones. Three pieces come out:

- the **prior** $P(y = c)$, how common the class is, which is just a count;
- the **evidence** $p(x)$, which is the same for every class and so cannot change which class wins;
- the **likelihood** $p(x \mid y = c)$: how often pumps of that class show this exact combination of readings — healthy pumps near 42 Hz and 5.6 bar, faulty ones all around.

The third is where all the difficulty sits. To estimate it you count pumps of that class near x. With two readings there are some: 1,200 pumps on a plane. With twenty readings the space is so large that near any combination there is almost no pump. This is the curse of dimensionality again, arriving from a different direction: before, it broke distances; here, it breaks the estimation of a density.

Slide 27 — Bayes' rule, applied to a class

$$P(y = c \mid x) = \frac{p(x \mid y = c)\, P(y = c)}{p(x)}$$

Read the left side as "what we want" and the right side as "what the data can give": how often pumps of class c produce readings like these, times how common class c is, divided by how common readings like these are overall. The denominator does not contain c, so it is the same number whether we are scoring "healthy" or "faulty".

$P(y = c \mid x)$ is what any probabilistic classifier wants, not only this one. The difference is the route: logistic regression estimates it **directly**, one formula from readings to probability (lesson 4); Naive Bayes reaches it **indirectly**, through the reverse probability and the class frequencies.

Question — the readings are continuous: shouldn't these be densities?

Yes, and that is why two of the letters are lowercase. For a continuous reading the probability of one exact value is zero, so $p(x \mid y = c)$ and $p(x)$ are **densities**. The rule holds unchanged, and the left side is still a probability: the density units cancel between the top and the bottom, $p(x) = \sum_c p(x \mid y = c)\, P(y = c)$. One consequence students meet a few slides later: a density, unlike a probability, can exceed 1.

Slide 28 — Drop the denominator: arg max picks the class with the largest numerator

$$\hat{y} = \arg\max_c \, p(x \mid y = c)\, P(y = c)$$

To *choose* a class we only need to know which score is larger, and dividing both scores by the same positive number does not change which one is larger. So the classifier compares the numerators and picks the larger. **arg max** over c returns the class c that makes the score largest — the class, not the score (`np.argmax` in code). And the score is not a probability: it is a density times a probability.

A natural objection is that the probabilities are then lost. Nothing is lost: the denominator is the sum of the numerators over all the classes, so dividing each score by that sum gives the probabilities back exactly. Slide 36 does this with numbers. Where Naive Bayes does lose the meaning of its probabilities is inside the numerator, through the assumption on the next slide.

Worked example — one pump, two classes, with and without the denominator

Priors from the 1,200 pumps: $P(\text{faulty}) = 735/1200 = 0.6125$, $P(\text{healthy}) = 0.3875$. Suppose the readings x of one pump have likelihood 0.02 under faulty and 0.05 under healthy.

class	prior × likelihood	÷ $P(x)$ = posterior
faulty	$0.6125 \times 0.02 = 0.01225$	0.387
healthy	$0.3875 \times 0.05 = 0.01938$	**0.613**

$P(x) = 0.01225 + 0.01938 = 0.03163$, the same divisor for both rows. Dividing changes the numbers into probabilities but cannot change which is larger: the decision is already made in the middle column. Here the rarer class wins, because its likelihood is two and a half times larger.

Slide 29 — The assumption: independent, given the class

Given the class, the features are independent of one another. The assumption is this sentence alone; the product of the next slide is its consequence, and the bell shape of each factor (slide 31) is a second, separate choice.

What it buys, on the pumps: instead of counting faulty pumps near 48 Hz **and** 5.6 bar together, count faulty pumps near 48 Hz ignoring pressure, then near 5.6 bar ignoring vibration — plenty of pumps for each. One n-dimensional density becomes n one-dimensional densities, each estimated from the rows of one class. In scikit-learn the model is `GaussianNB()`; after `fit()`, the means it estimated are in `theta_` and the variances in `var_`.

Question — what does "independent given the class" mean, and why did we assume it for the pumps?

> *Relating to Naive Bayes, explain again why "given the class, the features are independent of each other": what does being independent mean, and why did we assume it in the first example?*

Independent means that knowing one variable tells you nothing about the other. For two features x_1 and x_2, independence means that learning the value of x_1 does not change what you expect for x_2; equivalently, the joint density is the product of the two separate ones.

Independent given the class is a different statement. It says that the equality holds *inside each class*:

$$p(x_1, x_2 \mid y = c) = p(x_1 \mid y = c)\, p(x_2 \mid y = c).$$

Inside the healthy pumps, knowing the vibration tells you nothing more about the pressure; inside the faulty pumps, the same. But across the whole dataset the two features may well be related, because both depend on the class. An everyday case: ice-cream sales and drownings rise together, because both rise in summer; within one season they are unrelated — dependent overall, independent given the season. Naive Bayes allows dependence that runs *through* the class, and forbids any other kind.

Why we assumed it for the pumps: we did not assume it because we believed it. We assumed it because it is what *defines* Naive Bayes, and it is what turns an impossible estimation problem into an easy one. Whether it is true for the pumps is a separate question, and the lesson measures it on slide 40. The answer is that it is false, and the model works anyway; slide 41 explains why.

Question — an example of features that really are independent given the class

Can you give me an example of features that really are independent given the class y?

The cleanest way to be sure is to build the data so that it is true by construction. Take electronic components, each either good or defective, and two measurements: whether the component runs hot when tested, and whether its packaging is scratched. Suppose the scratches come only from shipping, a cause that has nothing to do with the temperature. Then, among the defective components, knowing that one runs hot does not change the chance that its packaging is scratched. If 0.8 of defective components run hot and 0.3 of them have scratched packaging, the fraction that does both is $0.8 \times 0.3 = 0.24$, exactly the product.

The instructive part is what the *whole* dataset looks like. Plot all the components together and the two measurements appear related: where one is large you are probably looking at a defective part, and there the other is large too. Split the plot by class and the relation disappears. **Once you know the class, knowing x_1 no longer helps you predict x_2.**

Slide 30 — The assumption, written: a product of n densities

$$p(x \mid y = c) = \prod_{j=1}^{n} p(x_j \mid y = c)$$

On the left, one n-dimensional density; on the right, n one-dimensional ones. On the pumps $n = 2$: $p(x \mid \text{faulty}) = p(\text{vibration} \mid \text{faulty}) \times p(\text{pressure} \mid \text{faulty})$. The product equals the joint density only if the assumption holds; otherwise it is an approximation, and slides 40 and 41 measure how good. The difference is a counting argument: to estimate the left side directly you need enough examples to fill an n-dimensional space; to estimate the right side, enough to draw n separate histograms. That is the bargain: an assumption that is usually false, exchanged for an estimation problem that can actually be solved.

Worked example — what the assumption saves, counted

Take n yes/no readings per pump. Describing $P(x \mid y = c)$ exactly needs a probability for every combination of answers; under the naive assumption it needs one probability per reading:

readings n	joint distribution, per class	naive, per class
2	$2^2 - 1 = 3$	2
10	$2^{10} - 1 = 1,023$	10
30	$2^{30} - 1 = 1,073,741,823$	30

With 30 readings the joint table has over a billion cells per class, almost all of them never seen in any training set. The naive model needs 30 numbers, each estimated from every row of the class. That is the bargain: it is false, and it is learnable.

Slide 31 — Gaussian: each factor is a bell curve

$$p(x_j \mid y = c) = \frac{1}{\sqrt{2\pi\sigma_{jc}^2}} \exp\left[-\frac{(x_j - \mu_{jc})^2}{2\sigma_{jc}^2}\right]$$

The factorisation leaves one choice open: the shape of each one-dimensional density. **Gaussian** Naive Bayes answers "a bell curve", with its own centre μ_{jc} and its own width σ_{jc} for every feature j in every class c — computed for feature j on the pumps of class c only, hence the two indices. On the pumps that is four bells. The bell is a choice, separate from independence: for counts, such as words in an email, scikit-learn has `MultinomialNB`.

So training is not an optimisation at all. It computes the class priors, and a mean and a variance per feature per class, in one pass over the data: no gradient descent, no learning rate, no iterations. Then the density of a new pump is the height of each bell at its reading.

Slide 32 — Training computes ten numbers: the widths separate the classes

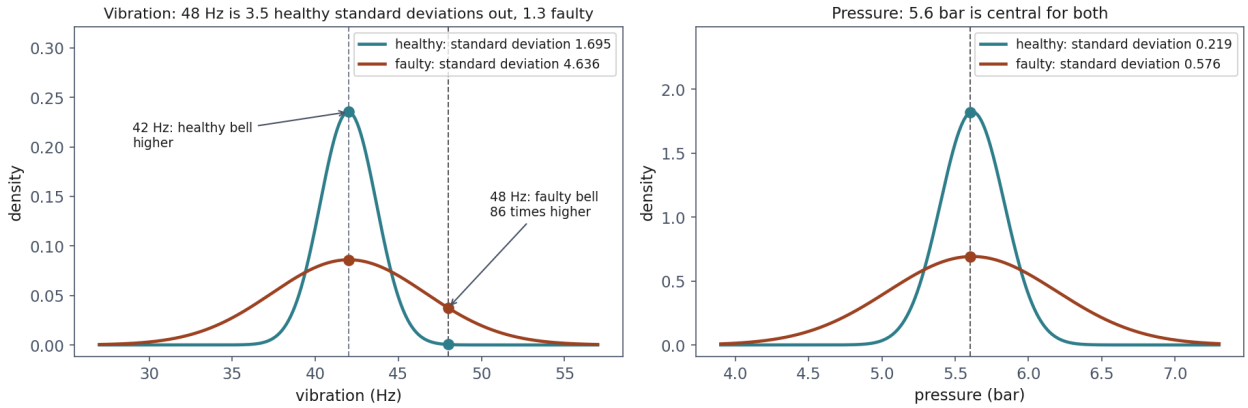
Class	Prior	Vibration mean	Vibration standard deviation	Pressure mean	Pressure standard deviation
Healthy	465 ÷ 1,200 = 0.3875	41.986 Hz	**1.695**	5.617 bar	**0.219**
Faulty	735 ÷ 1,200 = 0.6125	42.015 Hz	**4.636**	5.616 bar	**0.576**

Ten numbers: two priors, four means, four standard deviations, fitted on all 1,200 pumps. A "prior" is the class's share before looking at the readings. Read the means first: the two classes have **the same centre**, within a few hundredths of a hertz. What differs is the width, 1.695 against 4.636

Hz, 0.219 against 0.576 bar. Healthy pumps form a narrow bell around the design point, faulty pumps a wide one: the disc and the ring of slide 5, seen one feature at a time.

This explains two things at once. Logistic regression failed because a linear model separates classes by placing a boundary between their means, and here there is nothing between the means. And `f_classif` picked two noise columns on slide 23 for the same reason: it compares class means, and on these pumps they do not differ. Naive Bayes compares widths instead, and widths are where the information is.

Slide 33 — A pump at 48 Hz, 5.6 bar: there the faulty bell is 86 times higher



Follow the dashed line at 48 Hz up the left panel: the narrow healthy bell has fallen almost to nothing, the wide faulty one is still well up — 86 times higher. At 42 Hz the narrow bell is on top. On the right, 5.6 bar is central for both classes and barely discriminates; the narrow healthy bell peaks near 1.82, above 1.

Take a pump at 48 Hz and 5.6 bar. 48 Hz is 6 Hz from the design point. For the narrow healthy bell, standard deviation 1.695 Hz, that is three and a half standard deviations, far into the tail. For the wide faulty bell, standard deviation 4.636 Hz, it is barely more than one. So at 48 Hz the faulty bell is 86 times higher. At the design point itself, 42 Hz, the order reverses and the narrow bell is on top; that pump comes out healthy, with $P(\text{healthy}) = 0.820$.

On the pressure panel, 5.6 bar sits at the centre of both bells, so this reading barely discriminates. Note the height of the narrow healthy bell there: about 1.82, **greater than 1**. A density is not a probability: it is a probability *per unit* of the measurement, and a bell only 0.219 bar wide must rise high so that its total area is still 1.

Question — can the verdict be read here already, without the logarithm slides?

Yes, by multiplying three ratios, faulty over healthy. Vibration: the faulty bell is $0.0374/0.000433 = 86.4$ times the healthy one. Pressure: the healthy bell is the higher, $1.818/0.692 = 2.63$ times, so divide by 2.63. Class shares: $0.6125/0.3875 = 1.58$. Together

$$1.58 \times 86.4 \div 2.63 = 52 :$$

odds of 52 to 1 for faulty, so $P(\text{faulty}) = 52/53 = 0.981$. The ratio of the two products is the product of the three ratios, which is why the order does not matter. Each score being compared

is the numerator of Bayes' rule: the prior times the product of slide 30. Slides 34 to 37 reach the same 0.981 the way software does, in logarithms.

Slide 34 — Logs turn the product into a sum, and keep the same winner

$$\ln\left[P(y=c)\prod_j p(x_j \mid y=c)\right] = \ln P(y=c) + \sum_j \ln p(x_j \mid y=c)$$

Why take logarithms at all? With hundreds of features the product is hundreds of small densities multiplied together. A density can exceed 1, but far from a class's centre it is small, and the product becomes too small to store: a double cannot hold anything below about 10^{-308}, so two hundred factors of 0.01, 10^{-400}, are stored as exactly 0 (**underflow**), while the sum of their logarithms, -921, is an ordinary number. Every class would score zero and nothing could be compared.

Two facts repair this. The logarithm is increasing, so the class with the largest product is also the class with the largest logarithm: the value changes (-8.095 is not 0.000305) but the winner does not. And the logarithm of a product is the sum of the logarithms. This is the same reason lesson 4 worked with the log-likelihood. The logarithms are natural logarithms, so that e undoes them.

Question — is underflow "below machine precision"?

Not quite: two different limits. Machine precision, about 2.2×10^{-16}, is the relative gap between neighbouring doubles, and decides how many digits survive a rounding. Underflow is falling below the smallest number a double can hold, about 10^{-308}; that is the problem here.

Slide 35 — 48 Hz, 5.6 bar: scores and their logs

Class		Prior	Vibration density	Pressure density	Product, or sum of logs
Healthy	value	0.3875	0.000433	1.818	0.000305
Healthy	ln	-0.948	-7.745	$+0.598$	-8.095
Faulty	value	0.6125	0.0374	0.692	0.01585
Faulty	ln	-0.490	-3.286	-0.368	**-4.144**

This table is the whole of prediction. Read each class as two rows: the "value" row multiplies the prior by the height of that class's bell at each reading, and the "ln" row adds the same numbers after taking logarithms: $e^{-8.095} = 0.000305$. Faulty has the larger total, -4.144 against -8.095, so faulty is the prediction; the gap between them, 3.951, is $\ln 52$, the odds of slide 33. The healthy pressure entry is positive, $+0.598$, because the density 1.818 is above 1. No training row was consulted, only the ten stored numbers: the exact opposite of k-NN, for which prediction is the whole cost.

Question — where does each log-density come from?

From the bell of slide 31 in logarithms: $\ln p(x_j \mid c) = -\frac{1}{2}\ln(2\pi\sigma^2) - (x_j - \mu)^2/(2\sigma^2)$. For the faulty bell on vibration, $\mu = 42.015$, $\sigma = 4.636$, at 48 Hz:

$$-\tfrac{1}{2}\ln(2\pi \times 4.636^2) - \frac{(48 - 42.015)^2}{2 \times 4.636^2} = -2.453 - 0.833 = -3.286.$$

The first term is a charge for a wide bell, which spreads its density thinly; the second a penalty for being $5.985/4.636 = 1.29$ standard deviations from the centre. The same recipe on the healthy vibration cell, from the rounded table values, gives $-1.447 - 6.294 = -7.741$, not -7.745: the table uses the unrounded parameters, and that term, three and a half standard deviations out, is the most sensitive to the fourth decimal (handout section 4.2, Checks).

Slide 36 — From the two totals to probabilities

Step	Healthy	Faulty
1. Log total, previous slide	-8.095	-4.144
2. Undo the log: e^{total} = prior × densities	0.000305	0.01585
3. Divide by their sum: $0.01616 = p(x)$	**0.019**	**0.981**

Step 2 gives back the two numerators of Bayes' rule. Step 3 needs one fact: why is their sum equal to $p(x)$? Because a pump is either healthy or faulty, so its readings come from one of the two — the law of total probability:

$$p(x) = P(\text{healthy})\, p(x \mid \text{healthy}) + P(\text{faulty})\, p(x \mid \text{faulty}),$$

exactly the two scores added. Dividing each score by it gives probabilities that add to one: 0.019 and 0.981. This is the denominator that slide 28 dropped, recovered.

Question — so the model is 98% sure the pump is faulty: is that the truth?

It is the model's number. For this pump the truth is known: it sits $(48 - 42)/3.5 = 1.71$ steps from the design point, more than 1, so it is faulty by the rule, and labels are flipped at random 4% of the time, so the probability of seeing the label "faulty" is 0.96. The model says 0.981: a little more sure than the data allow, a first and small instance of slide 45.

Slide 37 — Software uses only the gap between the logs

The same result reached without computing the two scores, which is how software does it.

Question — how does one get from the gap to the formula 1/(1 + e^(−gap))?

In three steps. Write T_f for the faulty total, -4.144, and T_h for the healthy one, -8.095. Start from slide 36:

$$P(\text{faulty}) = \frac{e^{T_f}}{e^{T_f} + e^{T_h}}.$$

Divide top and bottom by e^{T_f}, which leaves the fraction unchanged: $1/(1 + e^{T_h}/e^{T_f})$. A ratio of exponentials is the exponential of the difference, $e^a/e^b = e^{a-b}$, so

$$P(\text{faulty}) = \frac{1}{1 + e^{T_h - T_f}} = \frac{1}{1 + e^{-\Delta}}, \qquad \Delta = T_f - T_h = 3.951.$$

With the numbers, $e^{-3.951} = 0.0192$, which is $0.000305/0.01585$, and $1/1.0192 = 0.981$.

Only the **gap** between the two totals matters, never the totals themselves, and that is the practical point. With hundreds of features the totals reach values like -800, e^{-800} is exactly zero in floating point, and the fraction of slide 36 becomes $0/0$; the gap stays an ordinary number. Read it as odds: $e^{3.951}$ is about 52, the 52 to 1 of slide 33. The formula $1/(1 + e^{-\Delta})$ is lesson 4's sigmoid.

Slide 38 — A cheap model, built on an assumption that is almost never true

The slide holds two halves, and both matter. The bargain is real: training is priors, means and standard deviations — counts, sums and sums of squares accumulated row by row, one pass — and an extra feature adds two numbers per class, its mean and its standard deviation, with nothing linking it to the others. That is why Naive Bayes can handle ten thousand features on a few thousand rows.

The assumption is false almost everywhere. On a real pump, vibration and pressure both depend on how hard it runs. In text, the words "New" and "York" appear together far more often than independence would allow. In medicine, the symptoms of one disease come in clusters. So the useful question is not "does the assumption hold?" but "**when does being wrong about it cost nothing?**" A *false assumption* and a *wrong classifier* are different things: to classify, the model only needs the right class to come out on top, not the probabilities to be correct.

Question — what exactly is the assumption on the pumps, with only two readings?

That once the class is known, the vibration says nothing about the pressure: $p(\text{vibration}, \text{pressure} \mid c) = p(\text{vibration} \mid c) \times p(\text{pressure} \mid c)$ for each class. The lesson's synthetic pumps have no "how hard it runs" driver — radius and angle are drawn independently — but within a class the readings still depend on each other, through the shape. By the rule, leaving the flipped labels aside, a healthy pump at 42 Hz can have any pressure from 5.15 to 6.05 bar, while a healthy pump at 45 Hz, already 3/3.5 of the way to the edge on vibration, only from 5.37 to 5.83 bar. Knowing the vibration changes the range the pressure can take: that is dependence (handout section 4.3).

Slide 39 — Naive Bayes on the pumps: 0.933

Model	Accuracy, 5 folds	Standard deviation over the folds
Majority baseline	0.613	0.000
Logistic regression	0.613	0.000
Gaussian Naive Bayes	**0.933**	0.010
k-NN, k = 5	0.944	0.006

A point behind k-NN, and a very long way ahead of the linear model, from a method that trains in a single pass and rests on an assumption we have just called almost never true. The right reaction is not to celebrate but to ask *why* it worked: a model that works because its assumption holds will keep working on similar data; one that works *despite* its assumption failing may not.

Question — is k-NN better than Naive Bayes, then?

Not on this evidence. The gap is 0.011, about 13 pumps of 1,200, and the fold standard deviations are 0.010 and 0.006: about one fold standard deviation, so call the two comparable rather than ranked. And $k = 5$ was chosen as the best k on these same folds, which makes its 0.944 slightly optimistic (lesson 5, nested cross-validation), while Naive Bayes had nothing to tune. Both point the same way: here Naive Bayes is as good as k-NN, at a fraction of the cost.

Question — can a good accuracy tell me that the assumption was true?

> *So in theory I could always take the Naive Bayes assumption as true, compute an accuracy, and from the result deduce whether the assumption was really satisfied, correct?*

The first half is right, the second is not. You can use Naive Bayes without knowing whether its assumption holds; that is normal practice. But **the accuracy cannot tell you whether the assumption holds**, and the pumps are built precisely to show it: accuracy 0.933, and an assumption that slide 40 proves false. The model predicts the class with the largest score; its estimated densities can be badly wrong while the class with the largest score is still the correct one. So there are two separate questions, answered by two separate tools: cross-validation tells you whether the model predicts well; only a study of how the features relate *within each class* tells you whether the assumption is true. A low accuracy does not prove the assumption false either.

Slide 40 — Uncorrelated within each class, but not independent

Correlation, within one class	between vibration and pressure	between the two standardised readings, squared
healthy pumps	−0.006	+0.168
faulty pumps	−0.049	**−0.420**

The middle column suggests the assumption holds: within each class the two readings are uncorrelated. But **correlation only detects straight-line dependence**. The faulty pumps fill a ring around the design point. On a ring, a pump whose vibration is far out must have a pressure close to the centre, otherwise it would lie outside the ring. That is a dependence, but a symmetric one: far out to the left and far out to the right both go with a central pressure, so a straight-line summary

averages it to zero. Square each reading's distance from the centre and it appears: −0.420 among the faulty pumps. The dependence is statistical, not mechanical: changing one reading does not change the other; knowing one changes what to expect of the other. Uncorrelated, yes; independent, no.

Question — how is the −0.420 computed?

In three steps (notebook 2). Standardise each reading with the mean and standard deviation of all 1,200 pumps: 42.004 Hz and 3.78 Hz, 5.616 bar and 0.471 bar. Square it, which keeps "how far from the centre" and forgets the side. Then take the ordinary correlation of the two squared columns within one class. One faulty pump, at 35.561 Hz and 5.233 bar: $z_1 = (35.561 - 42.004)/3.78 = -1.70$, squared 2.90; $z_2 = (5.233 - 5.616)/0.471 = -0.81$, squared 0.66. Over the 735 faulty pumps the two squared columns correlate at −0.420. Read it by quarters: faulty pumps whose vibration is near the centre (squared below 0.32) have a mean squared pressure of 2.38 — far out, to escape the hole — while those whose vibration is far out (above 2.34) have 0.64. One large, the other small: a negative correlation.

Question — and the healthy +0.168?

It is not the disc's own dependence. In a disc a large z_1^2 forces a small z_2^2, so the disc's own sign is negative. The +0.168 comes from the 28 pumps labelled healthy whose labels were flipped: truly faulty, far out in both readings. Leave them out and the 437 healthy pumps left give −0.036, and the faulty −0.420 becomes −0.460 without its 21 flipped pumps. The faulty number carries the argument.

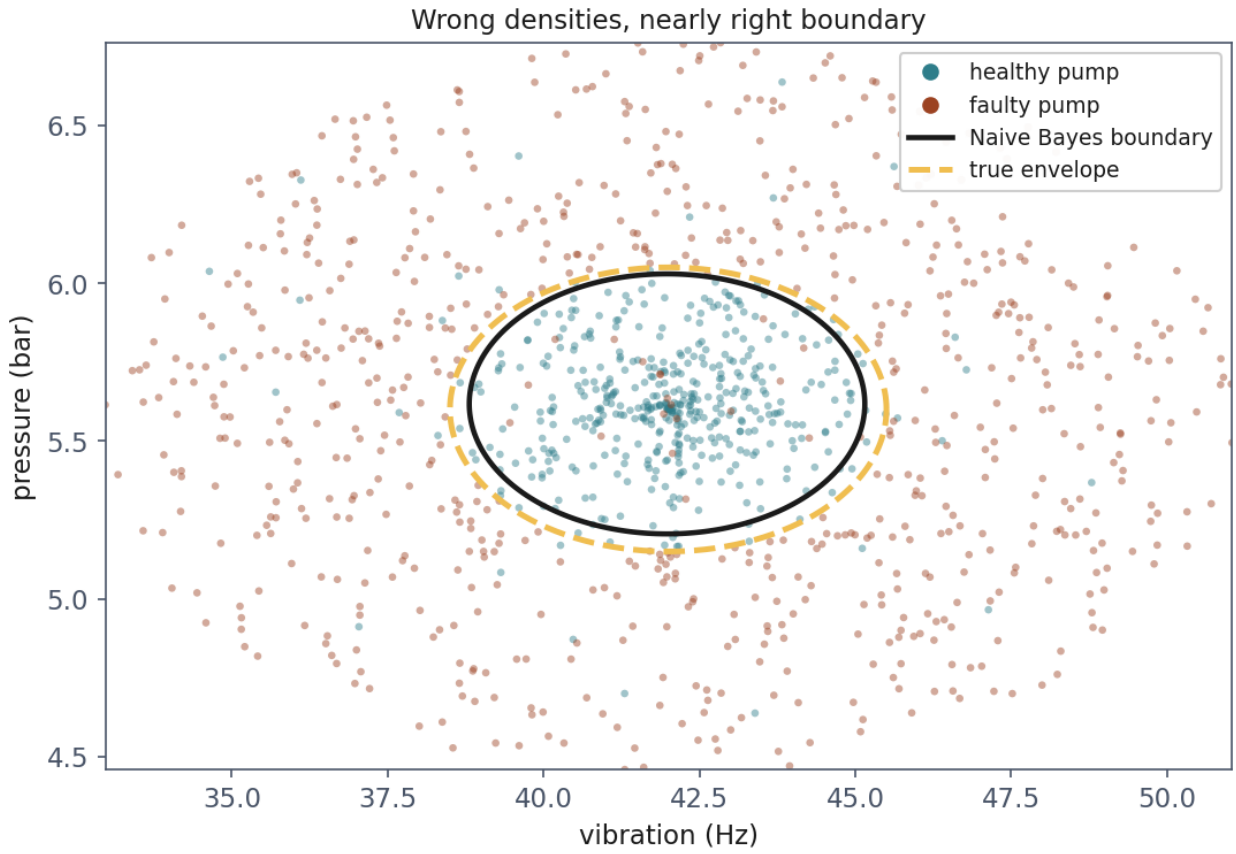
Question — why the square, and not the cube?

Because the dependence is about *how far* from the centre, not *which side*, so the transformation must treat −2 and +2 alike: it must be an **even** function. The square and the absolute value are even (the absolute value gives −0.454); the cube keeps the sign, so the two sides still cancel, and among the faulty pumps it finds only −0.022.

Question — how would you check the assumption with many features?

With two features you can look at a scatter plot; with a hundred you cannot. Three levels of checking, from cheap to hard. First, within-class correlation matrices drawn as heatmaps: a large entry within one class is strong evidence of dependence, but a correlation near zero proves nothing, as the pumps just showed. Second, measures that see curved dependence, such as **mutual information**, which is zero exactly when two variables are independent. Third, the hard limit: even if every *pair* of features is independent within a class, three can be related as a group. So do not try to *certify* the assumption; look for evidence that it is badly broken, and ask whether the breakage damages what you need from the model. On the pumps it is broken and costs almost nothing; on the interacting sensors of slide 42 it ruins the model.

Slide 41 — The densities are wrong, but the boundary they draw is about right



The solid ellipse is where the fitted model changes its mind, the dashed one the envelope the pumps were generated from: same shape and orientation, the model's a little smaller. The densities are wrong; the boundary they imply is nearly right.

So why 0.933? The densities are wrong in shape: two bell-shaped hills where the data are a disc and a ring; the faulty hill even peaks in the hole, where there are no faulty pumps. But the boundary they imply has the right shape. With both classes centred on the same point, subtract the healthy log score from the faulty one: what is left is a sum of terms $a_j(x_j-\mu_j)^2$ plus a constant, with every a_j positive because the healthy bell is the narrower. The faulty score gains on the healthy one as a pump moves away from the centre in any direction, and the boundary, where the two are equal, is an ellipse with axes along vibration and pressure — the shape of the true envelope.

Question — where do 3.17 Hz and 0.41 bar come from?

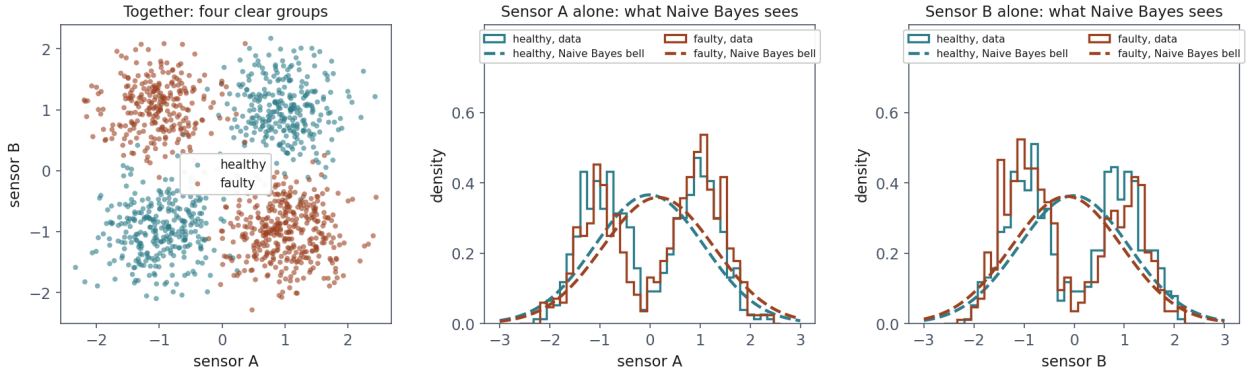
From the ten numbers of slide 32, with one centre for both classes (handout section 4.3): $a_{\text{vib}} = 1/(2\times 1.695^2) - 1/(2\times 4.636^2) = 0.1508$; $a_{\text{pres}} = 1/(2\times 0.219^2) - 1/(2\times 0.576^2) = 8.918$; the constant $b = \ln(0.6125/0.3875) + \ln(1.695\times 0.219/(4.636\times 0.576)) = 0.458 - 1.973 = -1.515$. The semi-axes, where $a_j d_j^2 = -b$, are $\sqrt{1.515/0.1508} = 3.17$ Hz and $\sqrt{1.515/8.918} = 0.41$ bar, against a true envelope of 3.5 by 0.45: the same proportions, about a tenth smaller. Densities are not "calibrated" or not — they are wrong in shape; calibration is about the probabilities, and it comes on slide 45.

Slide 42 — A second dataset: the signal is an interaction

A second pair of sensors on the same fleet. Each sensor is either high, around +1, or low, around −1, for half the pumps each; the reading is that value plus noise with a standard deviation of 0.45. A pump is faulty when **exactly one** sensor is high: the **exclusive or**, XOR. Both high is the designed high-load mode, both low is idle, and one without the other is a mismatch between demand and delivery. 628 of the 1,200 are faulty. The label follows each sensor's hidden state, not the reading, and no label is flipped here.

Ask what each sensor says *on its own* under this rule. The answer is nothing: whether a high A means "faulty" depends entirely on B. That is what an **interaction** is: the information lives in the combination of the features, not in any one of them, and it is exactly the structure that the independence assumption cannot represent.

Slide 43 — Each sensor alone: the two classes look the same



Left: together, the two sensors show four clear groups, each class owning two opposite quadrants. Middle and right: what Naive Bayes gets to see — each sensor alone. Solid outlines are the data, two humps each and the same two humps for both classes; dashed curves are the single bell per class that Naive Bayes fits, the two almost on top of each other.

On the left, four clean groups: any method that can draw a curved boundary will find the rule. In the middle and on the right, each sensor alone, which is all Naive Bayes ever looks at. The solid outlines are histograms of the data, with two humps each (low around −1, high around +1), and healthy and faulty have the *same* two humps, because each class is half low, half high on each sensor alone. A single bell cannot follow two humps, so it sits in the valley between them: centre near 0 and standard deviation about 1.1 for all four bells. On the pumps the widths carried the class; here nothing does.

Question — is the bell the culprit?

No, and that is the key point. Even the exact per-sensor densities, the two-hump histograms, are the same for the two classes, so a better shape per sensor would not help. What fails is the factorisation, which throws away the relationship between A and B before the model ever sees it. That is why no amount of data repairs it.

Question — what do the interacting sensors' numbers show?

> *1200 pumps, 628 faulty (52.3%). How much each sensor tells you on its own: correlation of sensor_a with the label +0.081, of sensor_b −0.064. And the means per class: healthy −0.012 and −0.006, faulty 0.167 and −0.149.*

They show, in numbers, what the middle and right panels show in pictures: each sensor on its own carries almost no information about the class. Always answering "faulty" scores 0.523. The correlations of each sensor with the label, +0.081 and −0.064, are close to zero, and the class means differ by 0.18 on sensor A and 0.14 on sensor B, against a spread of about 1.1: a product of this finite sample, not of the rule. Count the pumps in each quadrant instead — A high and B high, 273 healthy and 9 faulty; A high and B low, 10 healthy and 359 faulty; A low and B high, 11 healthy and 253 faulty; A low and B low, 278 healthy and 7 faulty — and the two sensors together classify almost perfectly. The 37 pumps in the wrong quadrant are reading noise, not flipped labels: a sensor in the low state sometimes reads above 0. Within each class the two sensors are strongly tied, +0.821 among the healthy pumps and −0.826 among the faulty: the assumption fails as badly as it can.

Worked example — the interaction, counted

100 pumps, 25 in each cell. A pump is faulty when **exactly one** reading is high:

	pressure low	pressure high
vibration low	25 healthy	25 faulty
vibration high	25 faulty	25 healthy

What Naive Bayes estimates is one column at a time, within each class:

- $P(\text{vibration high} \mid \text{faulty}) = 25/50 = 0.5$, and $\mid$ healthy also 0.5;
- $P(\text{pressure high} \mid \text{faulty}) = 0.5$, and $\mid$ healthy also 0.5.

Every factor is the same for both classes, so every pump gets the same score and the model can only fall back on the prior. The rule lives entirely in the combination, and the combination is exactly what the assumption throws away.

Slide 44 — 0.404: below the baseline, and below chance

Model	Accuracy
Majority baseline	0.523
Gaussian Naive Bayes	**0.404**
Logistic regression	0.393
SVM, linear kernel	0.606
k-NN, k = 5	0.967
SVM, RBF kernel	0.972

First, the problem is not hard: k-NN and the SVM with a radial basis function (RBF) kernel reach 0.967 and 0.972. It is hard only for a model that looks at one feature at a time. Second, 0.404 is

below chance: below the majority baseline of 0.523, and below a coin flip. The independence the model assumes is between the two *sensors*, given the class; here the sensors are anything but.

Question — how can a model do worse than guessing?

Because it does not know it has no information. The small gaps between the class means, 0.18 and 0.14, are the only per-feature evidence it has, so it follows them, and here they point the wrong way. Quadrant by quadrant, with the cross-validated predictions (notebook 2), the boundary is close to a line: of the 369 pumps with A high and B low it gets 359 right, every faulty one; of the 264 with A low and B high it gets 9 right, every faulty one missed; and it calls 239 of the 282 pumps with both high, and 213 of the 285 with both low, faulty, when nearly all are healthy. $42 + 359 + 9 + 75 = 485$ right of 1,200: 0.404. **A model with no signal does not sit politely at 50%: it follows whatever accidental structure it can find.**

Question — then invert every prediction, and it scores 0.596?

On this test, yes, $1 - 0.404$. But that is a choice made after seeing the validation results, a selection step that would itself need honest evaluation, and the inverted model still cannot represent the interaction. It would follow the next sample's accidents just as blindly.

Question — could a linear model solve it with larger coefficients, or another penalty?

No. The faulty pumps sit in two opposite quadrants; one straight boundary cannot cut those two off from the other two, whatever its coefficients. Lessons 3 and 4 tuned coefficients and penalties to improve a line; here the line itself is the wrong shape.

Slide 45 — Its probabilities cannot be trusted

The setting: the interacting sensors, Naive Bayes fitted on 900 pumps and tested on the other 300. For each pump it gives $P(\text{faulty})$ and predicts the larger class. Its **confidence** is the probability of the class it predicts, $\max(P, 1 - P)$, between 0.5 (a coin) and 1 (certain). A prediction at 0.51 is barely sure; one at 0.99 is very sure.

Two separate failures show up, and it pays to keep them apart:

- **Confidence does not tell right from wrong.** Mean confidence when right 0.567, when wrong 0.555. A useful model is surer when it is right, so you can trust the sure answers and check the unsure ones; this one gives you nothing to sort by.
- **The probabilities are not calibrated.** It is right on 133 of the 300, 0.443, while its mean confidence is 0.560: it says "56% sure" and is right 44% of the time. A calibrated model that says 56% is right about 56% of the time.

The more common complaint runs the other way: when several features carry the *same* evidence, near-copies such as ten words that always appear together, multiplying their probabilities counts that evidence ten times and pushes the posterior towards 0 or 1. Either way: **ranking and calibration are different properties**.

Question — is this the same distinction as the area under the curve against calibration?

> *When you say "ranking and calibration are different properties", is it the same point we made with the area under the curve (AUC) for logistic regression? AUC can be high while the model produces uncalibrated probabilities, correct?*

Yes, it is lesson 4's distinction (its Section 8). **Ranking**, also called discrimination: does the model place the positive examples above the negative ones? The area under the receiver operating characteristic (ROC) curve, AUC, measures this, and only this. **Calibration**: when the model says 0.8, are about 80% of those cases really positive? Take four examples, two negative and two positive. Model A gives them 0.10, 0.20, 0.70 and 0.90; model B gives 0.40, 0.45, 0.55 and 0.60. Both put the four in the same order, so both have an AUC of 1; but model B's numbers are timid. If all you need is an order (which pumps to inspect first), the ranking is what matters. If a decision depends on the probability itself, as in lesson 4's cost-based threshold, the numbers must be calibrated, and Naive Bayes's are not to be trusted.

Question — is accuracy a kind of ranking too?

A different one. The AUC ranks *pumps*: does a faulty pump score above a healthy one, over all thresholds? Accuracy here needs, for each pump, the right *class* on top: the gap between the two log scores on the right side of zero, or P on the right side of 0.5. Neither looks at calibration, and a model can be good at one and bad at the other.

Slide 46 — Notebook 2, live

The slide's table maps each section of the notebook to its slides and to the scikit-learn calls it uses. The notebook walks the whole story: Naive Bayes reaches 0.933 on the pumps, the within-class correlations look like zero, the correlations of the squared standardised readings show the dependence anyway (-0.420), the fitted ellipse explains why the cost is small, and the interacting sensors bring it to 0.404. The last row names the alternative: atoms 6 to 10 of the lesson 6 atoms notebook. The diagnostic to carry away: **measure the assumption, do not assume it.**

An experiment worth trying (it is atom 9): give Naive Bayes the product of the two sensors as a third column. The product is positive when both are high or both are low, negative when exactly one is high, so it carries the whole interaction in a single feature, and the model jumps from 0.404 to 0.972. That is the choice between engineering a feature and choosing a different model, in one cell.

Question — what do the notebook's conclusions mean?

> *Comment on the notebook's conclusions: "Very high dimensions with little data" (text classification), "As a baseline", "When the class ranking is what you need", and the one case to avoid it, "when the signal is an interaction".*

They answer a practical question: if the assumption is almost always false, when is Naive Bayes still the right choice? **Very high dimensions with little data**: documents described by the counts of fifty thousand words, three thousand training documents; estimating the joint density is hopeless, while one simple number per word per topic is easy, and a model that is roughly right and *can* be estimated beats a more faithful one the data cannot support (for counts the version

is `MultinomialNB`). **As a baseline**: training is a single pass with no tuning, so if a model that took hours of tuning does not beat it, you have learned something cheaply. **When the ranking is what you need**: an inspection list sorted by score works even though the scores are not real probabilities. **When to avoid it**: when the class depends on a *combination* of features, as on slides 42 to 44.

Part V — Support vector machines

Slide 47 — SVM: among the lines that fit, take the widest margin

When the two classes can be separated by a line, there are infinitely many lines that do it, and every one of them has zero training error. So the training error cannot choose among them: the count of errors is flat across all of them, with no slope to follow. Some other criterion must choose. Logistic regression chooses with the log loss. The support vector machine chooses differently: it takes the line with the **widest empty slab** around it, pushed out until it touches the nearest point of each class.

The reason is about generalisation, not fit. A boundary that passes close to a training point is one small change in the data away from misclassifying it. The boundary with the most room around it tolerates the most movement in the data before it changes its mind.

Question — what makes one line different from another, when both have zero training error?

The error counts only which side of the line a point is on, not how far from it. Lines differ in how close they pass to the nearest points. In one dimension, where the "line" is a threshold: healthy pumps at 1, 2 and 3, faulty ones at 7, 8 and 9. Every threshold between 3 and 7 makes zero errors. A threshold at 3.5 is 0.5 from the nearest healthy pump; one at 5 is 2 from both sides. A new healthy pump arrives at 4, one step from the others: 3.5 calls it faulty, 5 calls it healthy. The SVM picks 5, the middle of the empty gap, the threshold the data must move furthest to cross (atom 11 runs exactly this).

Question — why would gradient descent land on one line rather than another?

> *Why would gradient descent converge to the coefficients of one line rather than another? Where is the variability that makes it land on different lines?*

For the losses in this course there is none: each criterion picks one line. A loss that is convex with a single minimum — logistic regression with its penalty, scikit-learn's default, or the SVM's objective — has one answer, reached from any start. The two lines differ because the **objectives** differ (log loss against widest margin), not because of the process. Gradient descent is an **optimiser**: it finds what it is asked for. The SVM could be fitted by a gradient method too; what changes the line is the function being minimised, not the algorithm that minimises it. Variability of the kind the question imagines exists only for an algorithm that stops at the first line with zero errors, such as the perceptron, where the start and the order of the points decide where it stops.

The difference shows in what happens to a point that is already correctly classified. Write $y_i f(x_i)$ for how far point i sits on its own side, with labels ± 1 and $f(x) = w^\top x + b$. The log loss keeps falling as $y_i f(x_i)$ grows, so even a point that is safely correct keeps pulling on the boundary. The SVM's

hinge loss is exactly zero once $y_i f(x_i) \geq 1$, and the term $\frac{1}{2}\|w\|^2$ in its objective is what explicitly asks for the widest slab. (A curiosity: on separable data and with no penalty at all, logistic regression has no minimum — the loss keeps falling as the coefficients grow — and the *direction* of its coefficients drifts towards the widest-margin line. Nothing in a real fit relies on it.)

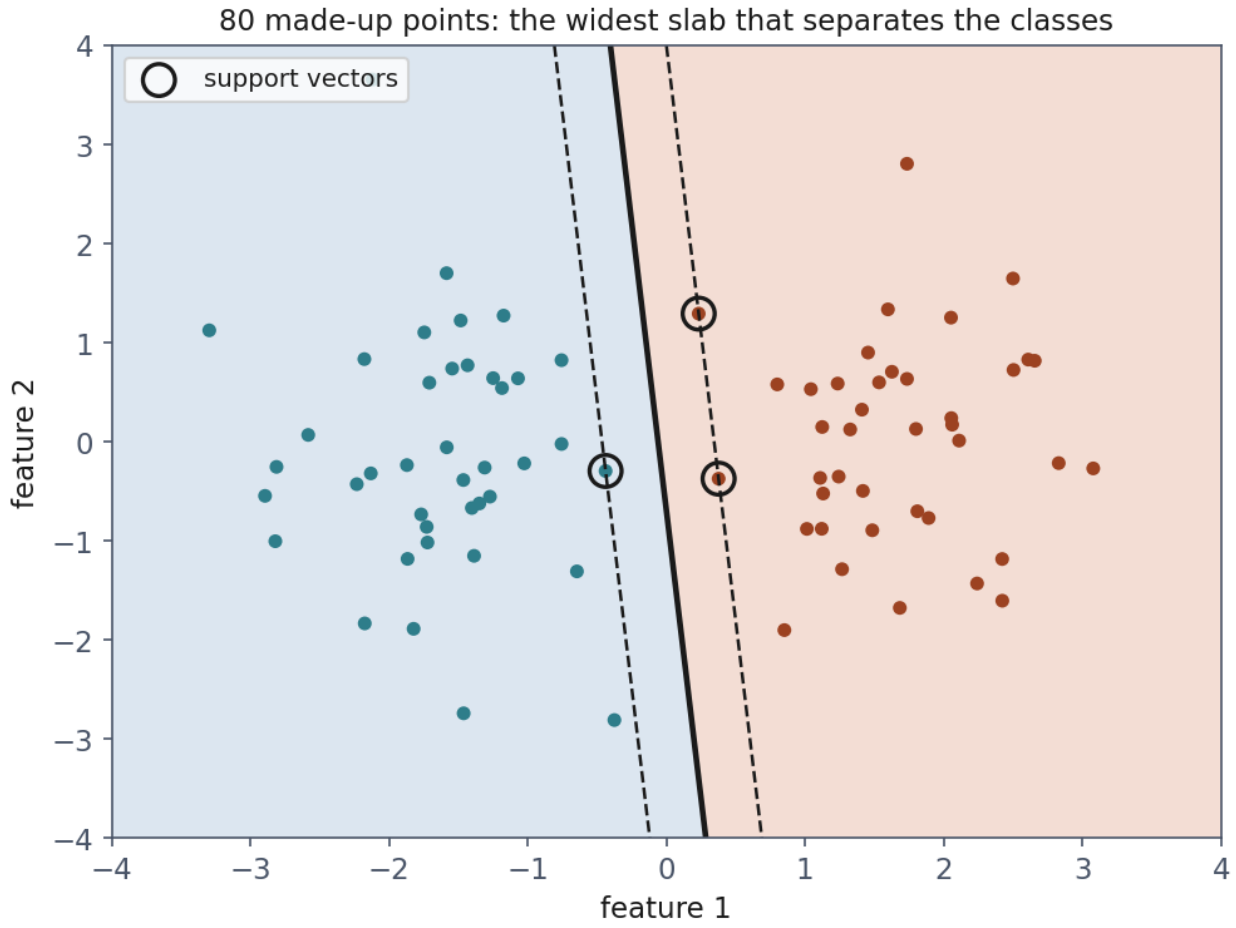
Worked example — distance to a boundary, and the width of the slab

Boundary $3x_1 + 4x_2 - 10 = 0$, so $w = (3, 4)$ and $\|w\| = 5$. A pump at $(2, 3)$:

$$\text{distance} = \frac{|3 \cdot 2 + 4 \cdot 3 - 10|}{5} = \frac{8}{5} = 1.6$$

If the closest pumps sit where $|w^\top x + b| = 1$, they are $1/5 = 0.2$ from the boundary and the slab is $2/\|w\| = 0.4$ wide. A boundary whose closest pumps were twice as far off, at 0.4, would satisfy the same rule only with $\|w\| = 1/0.4 = 2.5$, and its slab would be 0.8 wide. Under that rule a shorter w **is** a wider margin, which is why the SVM minimises $\|w\|$.

Slide 48 — 80 made-up points: three of them fix the line (support vectors)



80 made-up points, two groups of 40, and a linear SVM with a very large C, close to the strict problem of the next slide. The solid line is the boundary, the dashed lines the edges of the slab, and the circled points the support vectors touching it.

These are invented points, not pumps. Of eighty, three determine the boundary. Move any of the other seventy-seven anywhere, as long as it stays **outside the slab** (beyond the dashed line on its own side, not merely on its side of the boundary), and the boundary does not move: a point that enters the slab becomes a support vector and the line moves. These three are the **support vectors**: they support the boundary, the rest do not touch it. Training does not find them first; it solves for the line, and the support vectors are the points that end up on the edge of the slab.

This is new in the course. Every linear model of lessons 3 and 4 used every row, each contributing to the gradient. Two consequences. The fitted model is small: for a linear SVM, w and b are enough, three numbers here; a kernel SVM, later, must keep its support vectors, still not the whole dataset as k-NN does. And an outlier near the boundary matters a great deal while one far from it matters not at all.

Slide 49 — The strict problem: the widest slab with no point inside

The problem the previous figure solved, written down, so that the soft margin has something to relax. It is a slide the instructor may skip in class; the handout, section 5.2, derives each step.

- **Labels −1 and +1.** Lessons 3 and 4 used 0 and 1; here healthy is -1 and faulty $+1$, a relabelling with no content. It buys one product for "classified right": $f(x) = w^\top x + b$ is positive on the faulty side, so $y\,f(x) > 0$ says the sign of the prediction and the label agree, and $y\,f(x) \geq 1$ says the point is outside the slab. In code nothing changes: y stays 0/1 and `SVC` relabels internally; a positive `decision_function` means `classes_[1]`, here faulty.
- **The scale.** Multiplying w and b by ten does not move the boundary, so fix the scale by a choice: the closest points have $|w^\top x + b| = 1$. Then the slab reaches $1/\|w\|$ on each side, $2/\|w\|$ in all: a wide slab is a short w.
- **The problem.** Minimise $\frac{1}{2}\|w\|^2$ with every point outside the slab, $y_i(w^\top x_i + b) \geq 1$.

Question — what is the cost function here? With linear regression the cost was the MSE, and ridge added a penalty.

The parallel holds, with the roles swapped. In the strict problem the objective is **only** $\frac{1}{2}\|w\|^2$, the shape of ridge's penalty, and the data enter only as constraints, which forbid errors outright. The soft margin (next slides) adds the data term. And why require $y_i f(x_i) \geq 1$ rather than just > 0? Positive alone says the label is right but leaves the scale of w and b free; the 1 fixes it, so that the slab's edges are $f(x) = \pm 1$ and its width is $2/\|w\|$.

Slide 50 — Not separable? Charge for the violations

With 4% of the labels flipped, some faulty pumps sit among the healthy ones, and no slab can have all points outside it. The strict problem has no solution at all, not merely a bad one. So the SVM that is actually used everywhere is the **soft-margin** version: each point i gets an allowance $\xi_i \geq 0$

(read "xi"), its **slack**: how far it may fall short of its edge of the slab, in the units where the edge is at 1. The constraint becomes $y_i(w^\top x_i + b) \geq 1 - \xi_i$, and every unit of slack is charged at a price C.

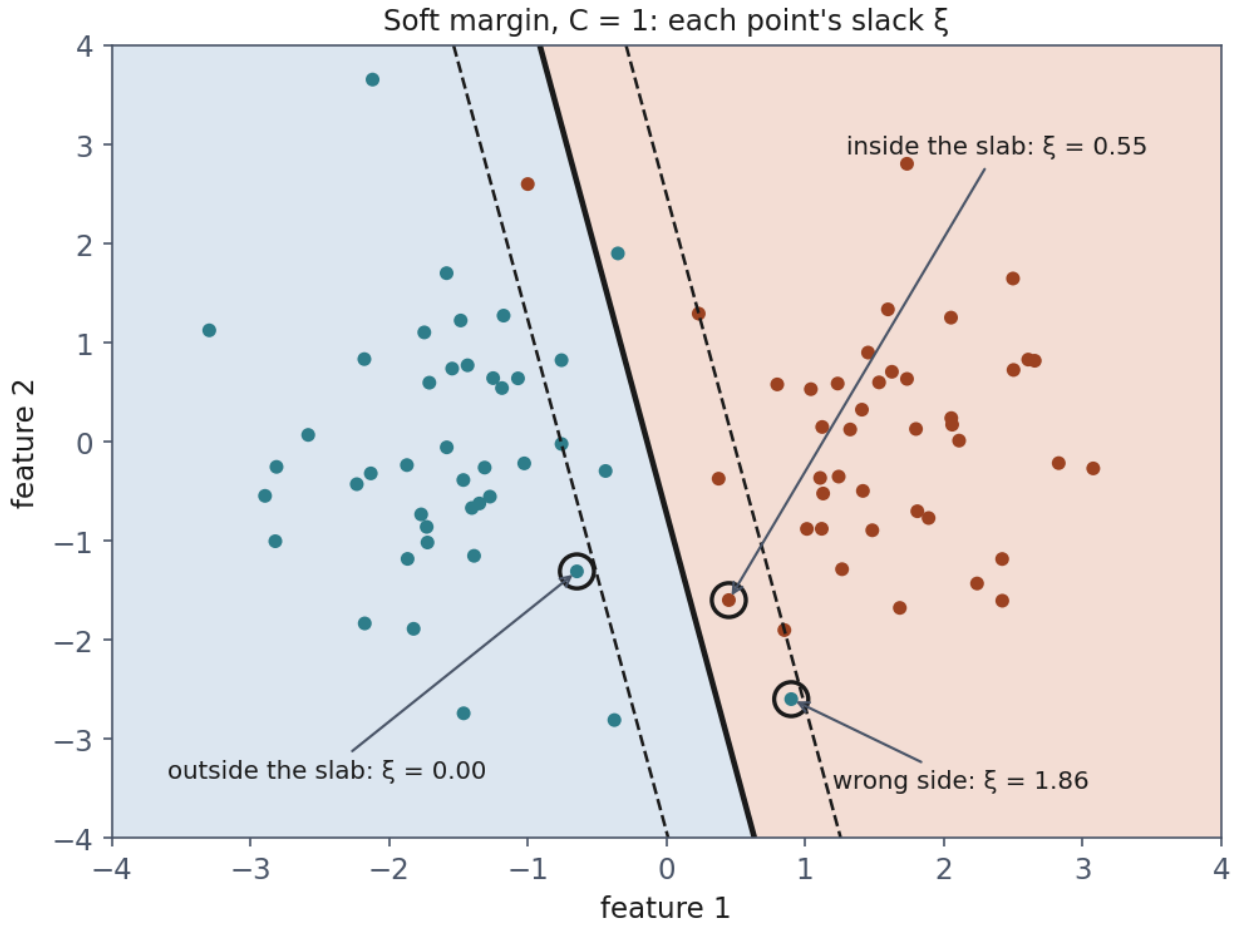
- $\xi_i = 0$: the point is outside the slab, on its own side, or exactly on its edge.
- $0 < \xi_i < 1$: inside the slab, but still on the correct side of the boundary.
- $\xi_i > 1$: on the wrong side of the boundary, so misclassified.

At the best fit each ξ_i is as small as its constraint allows, which is the hinge loss of the last bullet on the slide: $\xi_i = \max(0, 1 - y_i f(x_i))$. Three cases with numbers: a point with $y\,f(x) = 2$ has slack 0 and pays nothing; one with 0.4, inside the slab but on the right side, has slack 0.6 and pays $0.6\,C$; one with -0.5, on the wrong side, has slack 1.5 and pays $1.5\,C$. The trade: a wider slab lowers $\frac{1}{2}\|w\|^2$ but lets more points inside, raising $C\sum_i \xi_i$; C is the exchange rate.

Question — can a correctly classified point have non-zero slack?

Yes: when $0 < y_i f(x_i) < 1$ it is on its right side but inside the slab, and pays. The SVM charges not only wrong answers but right answers that sit too close to the boundary.

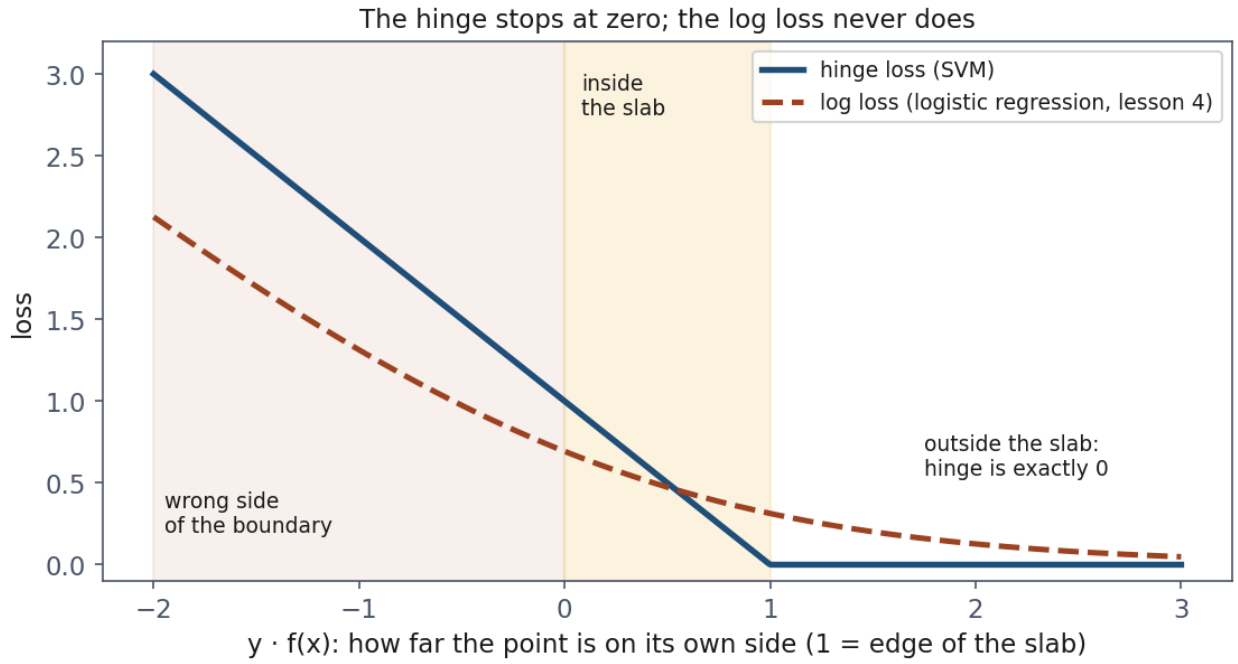
Slide 51 — Three kinds of point: outside the slab, inside it, on the wrong side



The 80 made-up points with four more placed against their class, and a soft-margin linear SVM with $C = 1$. Each circled point carries its slack.

The previous slide's three cases, drawn. The figure adds four points to the eighty of slide 48, placed against their class, and fits a soft-margin linear SVM with $C = 1$. Three points are circled (notebook 3 prints them): a teal point outside the slab on its own side, $y\,f(x) = 1.22$, slack 0, it pays nothing; a rust point inside the slab but still on the right side, $y\,f(x) = 0.45$, slack $1 - 0.45 = 0.55$; a teal point across the boundary, $y\,f(x) = -0.86$, slack $1 + 0.86 = 1.86$, more than 1 because it is on the wrong side. With $C = 1$ the slab is wider than on slide 48: errors are cheap enough that the model accepts a few points inside rather than squeezing the slab.

Slide 52 — The hinge loss stops at zero



Loss against $y \cdot f(x)$, how far a point is on its own side, 1 at the edge of the slab. Right of 1 the hinge is exactly zero, while the log loss keeps a small charge.

The horizontal axis is $y \cdot f(x)$ for one point: large and positive for a point safely on its own side, negative on the wrong side, 1 at the edge of the slab. (It is not "the margin": the margin is the slab's width, $2/\|w\|$, a property of the boundary, not of one point.) Left of zero, both losses are large. Between 0 and 1, inside the slab, the hinge still charges, falling along a straight line. Right of 1 the hinge is exactly zero, while lesson 4's log loss, rewritten for labels ± 1 as $\ln(1 + e^{-yf(x)})$, is still charging: 0.693 at 0, 0.127 at 2, 0.049 at 3. **Zero loss means zero gradient**: a point beyond 1 does not move w or b at all, and that is where support vectors come from. Under the log loss every point keeps a small pull, so logistic regression has no support vectors.

Worked example — five pumps, two losses

The margin of a pump is $y \cdot f(x)$: positive when it is classified correctly, larger the further it sits on its own side.

margin $yf(x)$	where it sits	hinge $\max(0, 1 - yf)$	log loss $\log(1 + e^{-yf})$
2.0	outside the slab, right side	**0.00**	0.127
1.0	on the edge	**0.00**	0.313
0.5	inside the slab	**0.50**	0.474
0.0	on the boundary	**1.00**	0.693
-1.0	wrong side	**2.00**	1.313

Past a margin of 1 the hinge is exactly zero: that pump stops influencing the boundary at all. The log loss never reaches zero, so every pump keeps a vote. That zero is where support vectors come from.

Slide 53 — The SVM objective: small w, plus C times the slack

$$\min_{w,b,\xi} \ \tfrac{1}{2}\|w\|^2 + C\sum_i \xi_i$$

subject to $y_i(w^\top x_i + b) \geq 1 - \xi_i$ and $\xi_i \geq 0$ for every training point. Read it against the strict problem of slide 49: the same $\frac{1}{2}\|w\|^2$, plus $C\sum_i \xi_i$, with the constraint relaxed from ≥ 1 to $\geq 1 - \xi_i$. The first term wants a wide slab; the second wants few and small violations; C sets the exchange rate.

Question — is this lesson 3's "loss plus penalty"?

Yes, once the slack is written as the hinge. Each ξ_i ends equal to point i's hinge loss, so dividing the whole objective by C, which does not change its minimum, gives

$$\sum_i \max(0,\ 1 - y_i(w^\top x_i + b)) + \frac{1}{2C}\|w\|^2 :$$

a loss on the data plus λ times a penalty on the coefficients, with the hinge in the place of the mean squared error, $\|w\|^2$ in the place of ridge's penalty, and $\lambda = 1/(2C)$. That one line explains the next slide's warning.

Question — so some points are accepted inside the margin, or even on the wrong side?

Yes, exactly. That is what "soft" means: the model is not required to classify every training point correctly, or even to keep every correct one outside the slab. Picture a single pump whose label was flipped to faulty, sitting deep among the healthy ones. To keep it outside the slab, the boundary would have to squeeze past it, leaving a very narrow slab for everyone else. The soft margin can instead accept a violation for that one point, pay $C\,\xi_i$ for it, and keep a wide, robust slab for the rest.

Slide 54 — C is the price of a training error

Large C makes violations expensive, so the slab narrows to classify more training points: low bias, high variance. Small C accepts some errors in exchange for a wider, calmer boundary. It is the third appearance of the same dial: λ in lesson 3, k earlier today, C now.

Mind the direction. A large λ means *more* regularisation; a large C means *less*, and the previous slide says why: divided by C, the penalty weighs $\lambda = 1/(2C)$. (k runs the other way from C: a larger k is smoother.) scikit-learn's `C` in `LogisticRegression` follows the same inverted convention.

Question — what happens at the two ends?

As C grows, every violation costs more. On data a line can separate, the strict problem comes back. On the pumps some violations are unavoidable whatever C is, so the soft problem stays solvable and only charges them more. As C falls towards 0, only $\frac{1}{2}\|w\|^2$ is left, its minimum is $w = 0$, and the model ignores the readings. And training accuracy need not rise steadily with C: the hinge also charges correct points inside the slab, so a larger C can trade an error for fewer slack points.

Question — in what sense is the margin "deformed"? Is it not always straight?

For a linear SVM it is always straight. The boundary $w^\top x + b = 0$ and the two edges of the slab, $w^\top x + b = \pm 1$, are three parallel lines, and they cannot bend. What a larger C can do is accept a larger $\|w\|$, which makes the slab **narrower**, and change w and b so that the lines **shift and rotate**. The image of a boundary that bends becomes accurate only with a kernel: with the radial basis function kernel the boundary is a flat plane in the lifted space, and drawn back in the original plane it can curve; a large C, especially with a large γ, lets it wrap around individual training points.

Worked example — two candidate boundaries, three prices

Boundary W is wide, $\|w\| = 1$, but two pumps violate it, with slack totalling 1.5. Boundary N is narrow, $\|w\| = 3$, and violates nothing. The SVM minimises $\frac{1}{2}\|w\|^2 + C\sum \xi_i$:

C	cost of W	cost of N	chosen
0.1	$0.5 + 0.1 \times 1.5 = 0.65$	$4.5 + 0 = 4.50$	**wide**
1	$0.5 + 1 \times 1.5 = 2.00$	$4.5 + 0 = 4.50$	**wide**
10	$0.5 + 10 \times 1.5 = 15.50$	$4.5 + 0 = 4.50$	**narrow**

Cheap violations buy the wide, calm boundary; expensive ones force the narrow boundary that bends to every training point. Large C means **less** regularisation — the direction that catches people out.

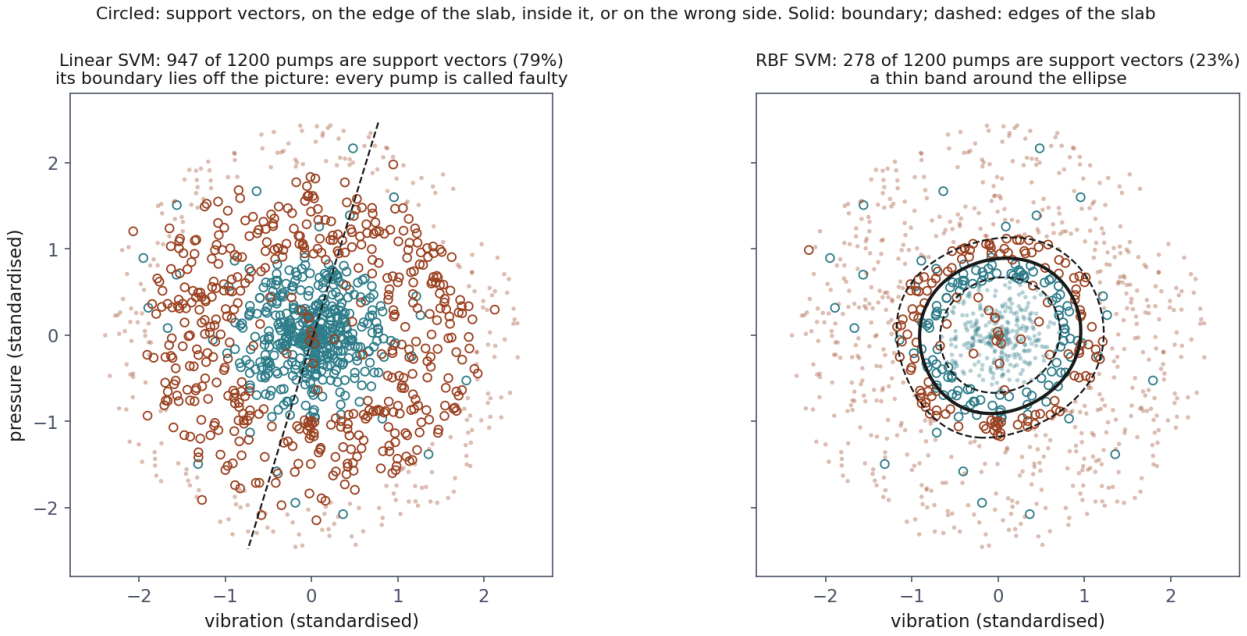
Slide 55 — The best straight line is still a straight line

Model, standardised pumps, default C = 1	Accuracy	Support vectors
SVM, linear kernel	**0.613 ± 0.000**	947 of 1,200 (**79%**)

Model, standardised pumps, default C = 1	Accuracy	Support vectors
SVM, RBF kernel	0.947 ± 0.005	278 of 1,200 (23%)

The linear SVM scores the base rate, exactly like logistic regression: on these pumps it too calls every pump faulty. The margin is a better way of *choosing among straight lines*; it does not give you anything other than a straight line. **A better criterion cannot rescue an inadequate family of models.** The same algorithm with one argument changed, `kernel="rbf"`, with scikit-learn's defaults ($\gamma = 0.5$, $C = 1$), reaches 0.947, the best number of the lesson. With slack, a support vector is any pump on the edge of the slab, inside it, or on the wrong side.

Slide 56 — On the pumps: a straight line leaves almost every pump in its slab



The two models on the standardised pumps, support vectors circled. Left, the linear SVM: its boundary lies off the picture and 947 of the 1,200 pumps sit in its slab. Right, the RBF SVM: 278 support vectors in a thin band around the ellipse.

The previous table's last column, drawn. Left, the linear SVM: its boundary lies off the picture, because it calls every pump faulty; the dashed line is the far edge of its slab, and almost every pump is inside it. Right, the RBF SVM: 278 support vectors in a thin band around the ellipse it draws; the pumps deep in the disc and far out in the ring are left uncircled, and the model forgets them.

Question — what exactly are the circled points?

The training pumps with non-zero weight in the fitted model: the ones on the edge of the slab, inside it, or on the wrong side. They are the only pumps the boundary depends on.

Slide 57 — Many support vectors: a free warning

If 79% of the training set is on or inside the slab, there is no empty space between the classes at all: the model has been forced to draw a boundary through a crowd. With the RBF kernel only 23% are. The diagnostic costs nothing: after fitting, compare `len(model.support_)` with the number of training rows, before any cross-validation. It also predicts the cost of prediction, since each support vector is one kernel evaluation per query.

Question — does a high share prove the kernel is wrong?

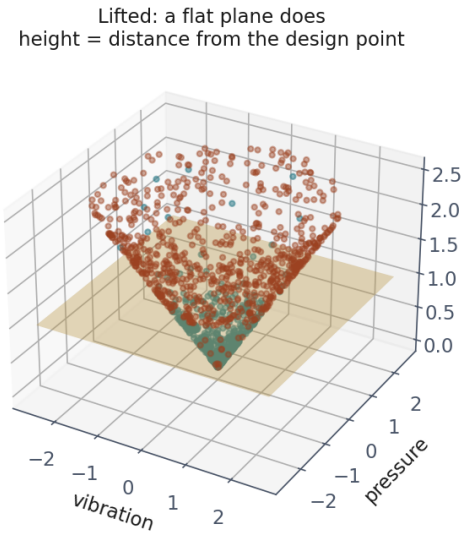
No; it is a warning, not a verdict. A kernel that cannot draw the shape the classes need raises it, as here, but so do classes that overlap, noisy labels, or the choice of C. The validation score and the boundary decide which. Nor does a smaller count mean a better model by itself: it describes how many examples constrain the solution, and the model's quality is still measured on held-out data.

Slide 58 — A new coordinate makes the classes separable

Take the scatter plot of slide 5 and add a third coordinate, computed from the two readings: each pump's distance from the design point, in standardised units (in raw units the hertz would swamp the bar). Lift every point to that height. Healthy pumps rise a little, faulty ones a lot, and a flat horizontal plane now separates them, apart from the flipped labels. **Separability is not a property of the data alone; it is a property of the data and the coordinates you measured it in**: the rule behind the classes was separable all along, in other coordinates. The unsatisfying part is that we chose this lift because we already knew the faults were radial. In a real problem you do not, and inventing the right coordinate is the hard part. That is the difficulty the kernel trick removes.

Slide 59 — The same pumps, lifted





The same 1,200 pumps twice: in the plane where they were measured, and lifted by their distance from the design point. The gold plane does what no line could.

Nothing was added: no new sensor. The third coordinate is a function of the two readings already there, so the lift gives the classifier no new information; it changes the shape of boundary a linear method can draw. The teal dots high up among the rust are the flipped labels. A flat cut up there is a circle down here: linear in the new coordinates is curved in the old ones. But this lift was three-dimensional and we could guess it. The spaces that work in general are enormous, sometimes infinite-dimensional; the next slides show that their coordinates never need computing.

Slide 60 — The SVM's solution: a weighted sum of pumps

The slide derives, in one step, the fact the kernel trick rests on: **the best w is not a new vector the optimisation invents, but a weighted sum of the training points themselves.**

Question — where does $w = \sum_i a_i y_i x_i$ come from? It looks dropped from above.

Write slide 53's objective with the hinge in place of the slack, $\frac{1}{2}\|w\|^2 + C\sum_i \max(0,\ 1 - y_i(w^\top x_i + b))$, and set its slope in w to zero. The first term contributes w. Each hinge term contributes $-C\,y_i x_i$ while pump i is inside the slab or on the wrong side, nothing once it is outside, where the hinge is flat, and something in between exactly on the edge, where the hinge has a corner. Zero slope gives

$$w = \sum_i a_i\, y_i\, x_i,$$

with $a_i = C$ for the pumps inside or on the wrong side, $a_i = 0$ outside, and $0 \le a_i \le C$ on the edge. So w is built from the pumps because the only pieces of the slope that involve data are the $y_i x_i$ of the pumps that push. The picture: the boundary is a tent held in place by the pegs pressing against it, and a_i is how hard pump i presses. On the pumps with the RBF kernel: 268 at the ceiling C, 10 in between, 922 at zero. (Most texts write α_i; this course keeps α for the learning rate.)

Question — what is the dual formulation, intuitively?

Primal and **dual** are two ways of stating the same optimisation. The primal finds w and b directly, one unknown per feature plus the intercept. The dual finds one weight a_i per *training point*, saying how much each point matters to the boundary. The long route to it attaches a price to each constraint (a multiplier, one per training point, paid only when that point pushes against its constraint), and the soft margin adds a second multiplier for each $\xi_i \ge 0$; their sum is C, which is where the ceiling $a_i \le C$ comes from (handout section 5.4). Substituting w back leaves a problem in the a_i alone, in which the training points appear only as $x_i \cdot x_j$. That last fact is what makes the kernel trick possible.

Question — were the 922 zero-weight pumps ignored during training?

No. All 1,200 pumps took part in the optimisation; the 922 at weight zero can be dropped only *after* it, because they add nothing to the prediction formula. Had one of them sat closer to the boundary, it would have pushed, and the answer would differ.

Slide 61 — So the new coordinates are never computed

Call $\phi(x)$ a pump's new coordinates — slide 58's (standardised vibration, standardised pressure, distance from the centre) is one choice. Run slide 60 in those coordinates and $w = \sum_i a_i y_i \phi(x_i)$. Put that w into the boundary and every $\phi(x_i)$ meets $\phi(x)$ only in an inner product:

$$f(x) = \sum_i a_i \, y_i \, \phi(x_i) \cdot \phi(x) + b.$$

Training is the same: f at each training pump and $\|w\|^2$ are both sums of $\phi(x_i) \cdot \phi(x_j)$. So a function that returns that inner product straight from the original two readings — a **kernel**, $K(x, x')$ — is all either computation needs, and ϕ is never computed. The radial basis function kernel, scikit-learn's default, does this for an infinite-dimensional space at the cost of one exponential per pair. (Kernels were joined to the margin by Bernhard Boser, Isabelle Guyon and Vladimir Vapnik at Bell Labs in 1992.)

Slide 62 — The kernel trick by hand: 25 = 25

Route, for u = (1, 2) and v = (3, 1)	Computation	Result
Lift, then multiply	(1, 2.83, 4) · (9, 4.24, 1) = 9 + 12 + 4	**25**
Never lift: the kernel (u · v)²	(1×3 + 2×1)² = 5²	**25**

The lift is $\phi(x) = (x_1^2,\ \sqrt{2}\,x_1x_2,\ x_2^2)$, and the two points are $u = (1, 2)$ and $v = (3, 1)$ — not a and b, which are already the weights and the intercept. First route: lift both, $\phi(u) = (1,\ 2\sqrt{2},\ 4)$ and $\phi(v) = (9,\ 3\sqrt{2},\ 1)$, and take their inner product: $9 + 2\sqrt{2} \times 3\sqrt{2} + 4 = 9 + 12 + 4 = 25$. Second route: never leave the plane, $u \cdot v = 1 \times 3 + 2 \times 1 = 5$, then square it: 25.

Question — why is it always the same number, and why the √2?

Expand the square for any two points:

$$(u{\cdot}v)^2 = (u_1v_1{+}u_2v_2)^2 = u_1^2v_1^2{+}2u_1v_1u_2v_2{+}u_2^2v_2^2 = (u_1^2)(v_1^2){+}(\sqrt{2}\,u_1u_2)(\sqrt{2}\,v_1v_2){+}(u_2^2)(v_2^2) = \phi(u){\cdot}\phi(v).$$

The $\sqrt{2}$ in ϕ is there to produce the 2 of the cross term. So the function $(u \cdot v)^2$ *is* the inner product in the three-dimensional space, computed without going there. And that space contains x_1^2 and x_2^2, whose sum is the squared distance from the centre: a flat plane there is a circle or an ellipse in the original plane, the lift slide 58 chose by hand.

Question — how do I find $K(x_i, x_j)$ without knowing $\phi(x_i) \cdot \phi(x_j)$?

You do not go from ϕ to K; in practice you go the other way. You **choose** K from a family of functions known to be inner products in *some* space, and the mathematics guarantees that the space exists; you never need to write it down. This slide goes from ϕ to K only to convince you that the mechanism works. For the radial basis function you just compute $\exp(-\gamma\|x - x'\|^2)$ from two ordinary vectors; that this equals an inner product in an infinite-dimensional space is a

theorem (handout section 5.4). Which functions qualify is **Mercer's condition**: for any set of points, the table of all values $K(x_i, x_j)$ must be symmetric and never produce a negative value of $\sum_i \sum_j c_i c_j K(x_i, x_j)$, whatever numbers c_i you choose. The standard kernels (linear, polynomial, radial basis function) satisfy it.

Slide 63 — The RBF kernel: similarity falls with distance

The **radial basis function (RBF) kernel**, $K(x, x') = \exp(-\gamma \|x - x'\|^2)$, is a **similarity**: 1 for two identical pumps, falling towards 0 as they move apart. On the standardised pumps with $\gamma = 0.5$: 0.61 at one standard deviation, 0.14 at two, 0.011 at three; it halves at $\sqrt{\ln 2/\gamma} = 1.18$ standard deviations. *Radial* because its value depends only on the distance, not on the direction; it is also called the Gaussian kernel, because $\exp(-\gamma d^2)$ is a bell in the distance d.

Why this kernel: expand the exponential and it holds $(x \cdot x')^2$, $(x \cdot x')^3$ and every other degree at once — slide 62's kernel is the degree-2 term — so its ϕ has infinitely many entries and can draw the circle the pumps need, or any smooth shape, without being told which. And it is a relative of k-NN: with this kernel the boundary is a vote of the support vectors weighted by similarity, near ones counting, far ones barely. The cost: one exponential per pair of points, so the kernel matrix has $m \times m$ entries, which is why SVMs become impractical above about a hundred thousand rows.

Question — does a similarity of 0.61 mean a 61% chance that two pumps share a class?

No. It is geometry, not probability: two pumps with identical readings have similarity 1 whatever their labels.

Question — is = 0.5 scikit-learn's fixed default?

No. The default is `gamma="scale"`, which computes 1/(number of columns × their variance); it comes out at 0.5 here because there are two columns and both are standardised. On other data the same default gives a different number.

Worked example — how similar two pumps are, by distance

$K(x, x') = e^{-\gamma\|x-x'\|^2}$ with scikit-learn's default on the standardised pumps, $\gamma = 0.5$:

distance (standard deviations)	similarity K
0.0	1.000
0.5	0.882
1.0	0.607
1.177 (half-way)	0.500
2.0	0.135
3.0	0.011

Identical pumps score 1; at 1.18 standard deviations the similarity has halved; at 3 it is about 1%. Every vote in the SVM's prediction is weighted by one of these numbers, so a pump three spreads away is, in effect, not consulted.

Slide 64 — Training: one weight per pump

Training finds the weights a_i of slide 60. It minimises slide 53's objective, but with the RBF kernel w lives in the infinite space and cannot be stored, so the same problem is solved in the weights instead (the dual, handout section 5.5): one weight per pump, $0 \le a_i \le C$, $\sum_i a_i y_i = 0$, needing only $K(x_i, x_j)$ between training pumps. The solver is not gradient descent: it starts with every weight at zero, picks **two** weights at a time and sets them to their best values with all the others fixed, then repeats until no pair improves. Two at a time, because the constraint $\sum_i a_i y_i = 0$ forbids moving one alone. The problem is convex, so there is one answer, found the same way every run, with no learning rate. This is sequential minimal optimisation (SMO), published by John Platt in 1998 and run inside the libsvm library that scikit-learn's `SVC` calls.

On the pumps the weights fall into the three groups of slide 50: **922 pumps at weight 0**, safely on their own side; **268 at the ceiling,** $a_i = C = 1$, inside the slab or on the wrong side; **10 on the edge**, with a weight between 0 and C. The 268 and the 10 are the 278 support vectors, and the stored model is those 278 points, their weights, and the intercept $b = +1.105$.

Question — who are x_i and x, during training and during prediction?

The prediction formula is $f(x) = \sum_i a_i\, y_i\, K(x_i, x) + b$: x_i and y_i are support vectors and their labels ($+1$ faulty, -1 healthy), and x is the **new pump**. During training there is no new pump yet: the solver uses only $K(x_i, x_j)$, where both are training points — the $m \times m$ kernel matrix. The same function, two different second arguments.

Slide 65 — The SVM's score at 48 Hz, 5.6 bar: Σ a y K(x , x) + b = +1.883

Term	Value
votes from faulty support vectors	+37.455

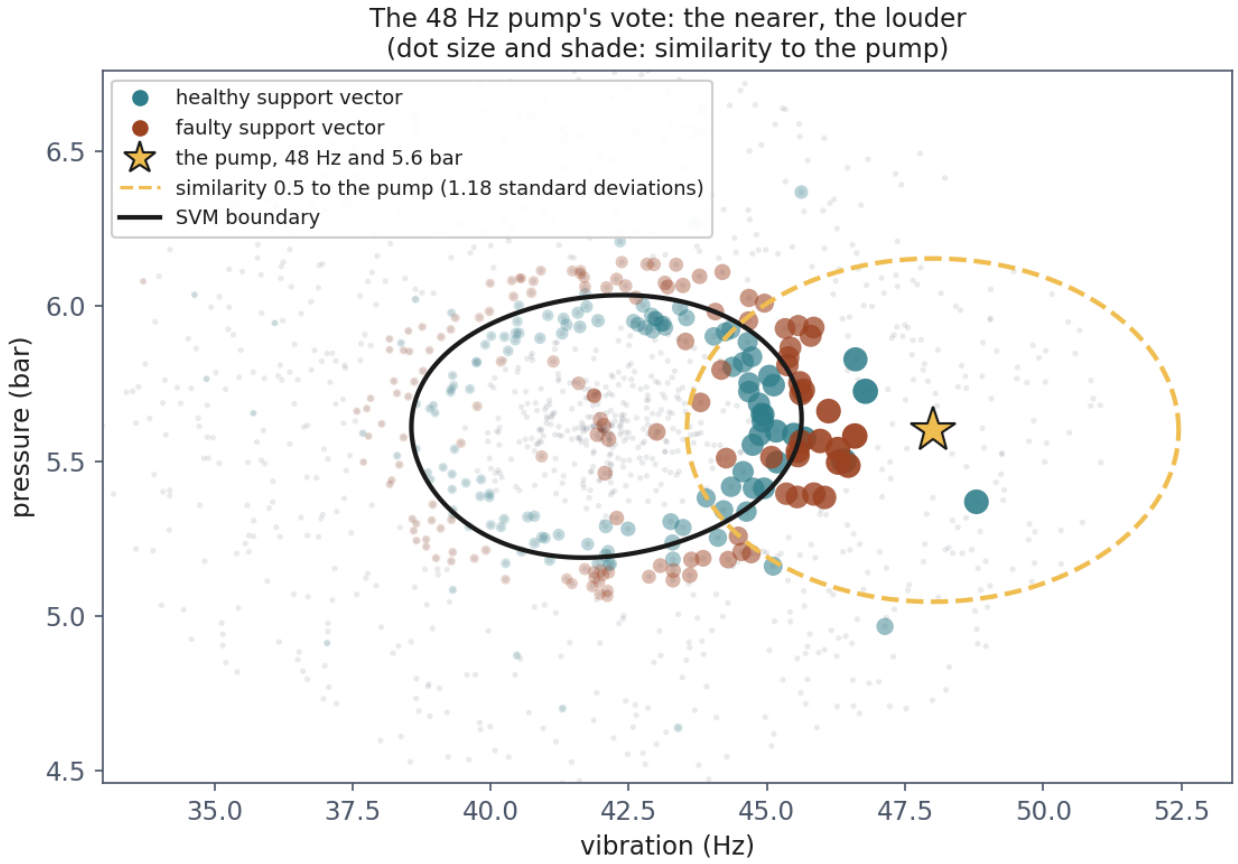
Term	Value
votes from healthy support vectors	-36.677
intercept b	$+1.105$
total	**+1.883**

The title is the formula: over the 278 support vectors, weight times label times similarity, plus b; its sign is the class. Four steps: standardise the pump with the training scaler, to $(1.587, -0.035)$; compute its RBF similarity to each support vector; add up weight $\times$ label $\times$ similarity; add b. With $C = 1$, 268 of the 278 weights are exactly 1, so most votes are just label times similarity. The two nearest support vectors make it concrete (notebook 3): a faulty pump at 46.59 Hz and 5.58 bar, similarity 0.932, votes $+0.932$; a healthy one at 46.78 Hz and 5.73 bar, similarity 0.916, votes -0.916. The total is exactly what `decision_function` returns: $+1.883$, faulty, the same answer Naive Bayes gave by a completely different route.

Question — how should the total be read?

The two big sums nearly cancel, so the neighbourhood decides: the 56 support vectors with similarity above one half contribute a net $+1.141$, the other 222 only -0.363. The intercept on its own, $+1.105$, is what a pump far from every support vector would score: faulty, sensible here, where far from the design point means faulty. And $+1.883$ is above 1: by slide 52's reading the pump sits beyond the faulty edge of the slab, faulty with room to spare. It is a score, not a probability.

Slide 66 — One pump's vote, drawn



Each support vector sized and shaded by its similarity to the pump (the star). The dashed line is where the similarity is one half.

The previous table as a picture. Each support vector is sized and shaded by its similarity to the star. The dashed ellipse is where similarity falls to one half: in standardised units a circle of radius 1.18, which on these raw axes stretches by each reading's standard deviation, $1.177 \times 3.78 = 4.45$ Hz across and $1.177 \times 0.471 = 0.55$ bar up and down. The 56 inside it carry most of the weight. Beside the star, teal and rust pumps lie together, pumps just outside the envelope and some flipped labels, which is why the nearest votes nearly cancel and the balance of the whole neighbourhood decides. In this sense the SVM is k-NN again, with a smooth similarity in place of a hard k, only the support vectors voting, and learned weights.

Slide 67 — Training against cross-validated, three settings

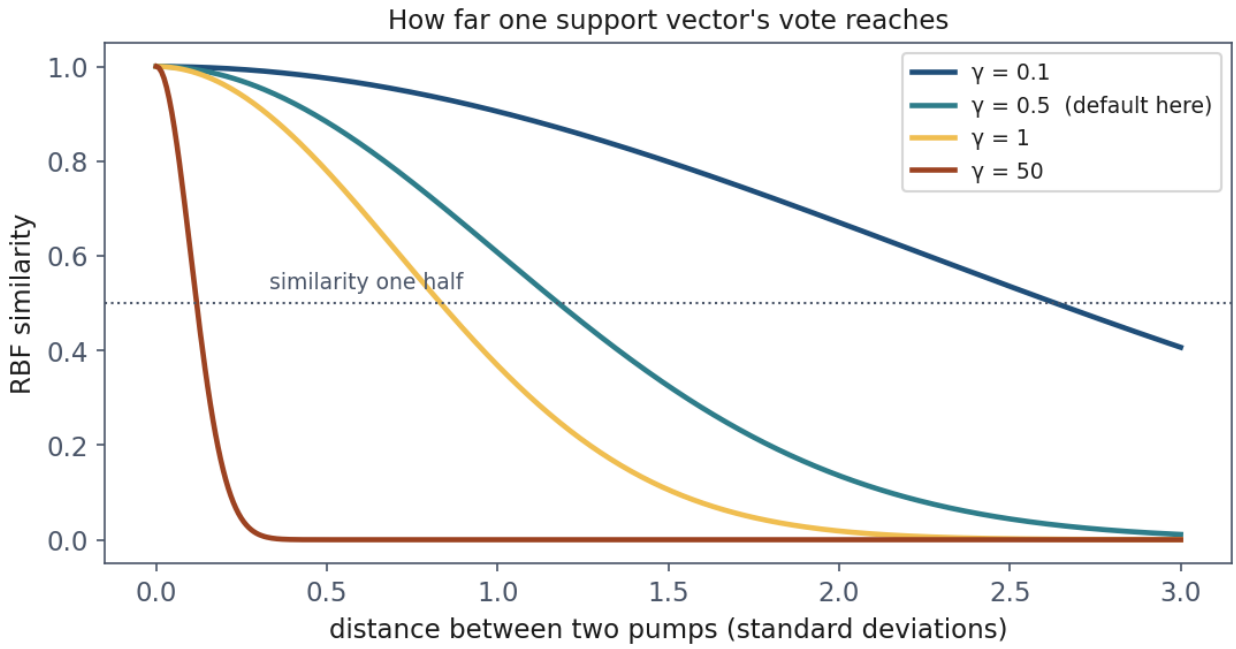
Setting	Training (each fold's 960)	Cross-validated
$= 0.1$, C $= 1$	0.934	0.929
$= 1$, C $= 1$	0.948	**0.944**
$= 50$, C $= 1000$	**0.996**	0.902

Both columns come from the same five folds, as for k on slide 12. The bottom row has the best training score and the worst cross-validated one: the signature of overfitting, measured on the same data by two procedures. Choosing on the training score would select the worst model. The instinct behind that choice is not foolish: everywhere else in engineering, doing better on the data you have is evidence of a better system, and lessons 3 and 4 spent their time fitting data well. What breaks the analogy is that a flexible model can fit data it has already seen without learning anything that transfers.

Question — is it or C that overfits?

Neither alone. Turned one at a time, each barely hurts (notebook 3): $\gamma = 50$ with $C = 1$ scores 0.960 training and 0.940 cross-validated; $\gamma = 1$ with $C = 1000$, 0.953 and 0.948. Together they overfit: a narrow reach and expensive errors buy an island around every flipped pump. And "best in the middle" holds among these settings only: the default $\gamma = 0.5$, $C = 1$ scores 0.947. Finding the best is a cross-validated grid over both, lesson 5's `GridSearchCV`, with nested cross-validation to report it honestly.

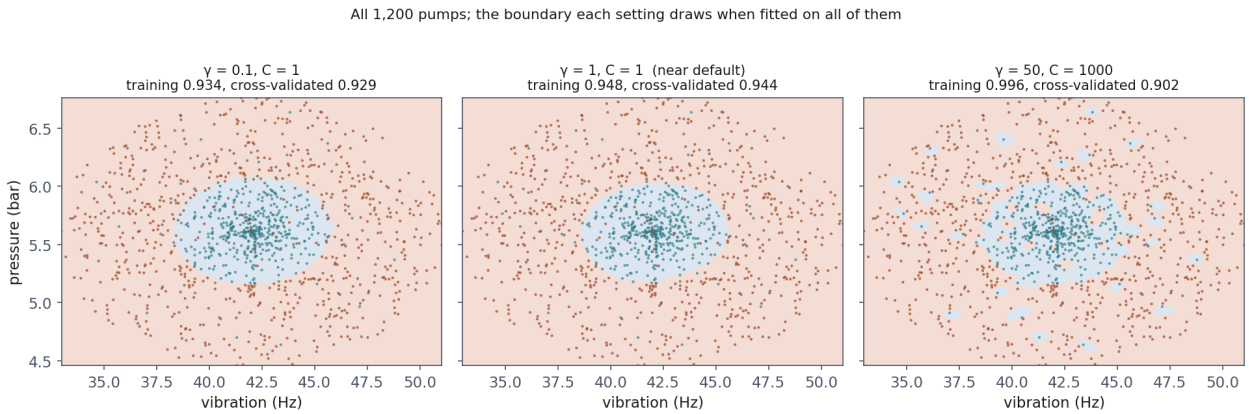
Slide 68 — How far one support vector's vote reaches, for four values of



RBF similarity against distance, one curve per γ. Where each crosses the dotted line at one half is its reach: 2.63, 1.18, 0.83 and 0.12 standard deviations.

γ turned into a distance. Where each curve crosses one half is the distance at which a support vector's vote has halved: 2.63 standard deviations at $\gamma = 0.1$ (the widest reach), 1.18 at the default 0.5, 0.83 at 1, and 0.12 at 50. The reach goes as $1/\sqrt{\gamma}$: a hundred times the γ is a tenth of the reach. In k-NN's terms, a small γ behaves like a large k, many pumps voting and a smooth boundary; a large γ like $k = 1$, only the nearest speaking. Before looking at the next slide, predict what the boundary will look like at $\gamma = 50$.

Slide 69 — Overfitting you can see: the boundary at the three settings



Three settings, with the training and cross-validated scores in each title; all 1,200 pumps drawn, so every island shows the pump it was drawn around.

Left, $\gamma = 0.1$: a smooth boundary, slightly too smooth, and the two scores agree. Middle, $\gamma = 1$: close to the true envelope, and the best cross-validated score. Right, $\gamma = 50$ with $C = 1000$: the boundary has broken into islands around individual pumps, both ways — teal islands in the faulty ring around healthy-labelled pumps far out, rust holes in the healthy disc around faulty-labelled pumps near the centre — mostly the flipped labels. That is what a training score of 0.996 looks like from outside: a private territory around each flipped label. A new pump landing between two islands is judged by whatever stray point is nearest, and that is where the 0.902 comes from. Since lesson 5 overfitting has been a curve on a plot; here it is a shape in the input space, visibly wrong. When you can plot the boundary, plot it.

Part VI — Closing

Slide 70 — Notebook 3, live

The slide's table maps each section of notebook 3 to its slides, the scikit-learn calls it uses, and the number to come back with; the last row offers atoms 11–15 of the lesson 6 atoms notebook in its place, in the same time. The notebook fits the linear and RBF SVMs on 80 made-up points and then on the pumps, counts their support vectors (947 against 278), checks the kernel trick by hand ($25 = 25$), takes one pump through the vote ($+1.883$), and sweeps γ and C with the boundary drawn each time. The step worth doing yourself is turning γ up and watching the islands form: it is the most direct experience of overfitting in the course. Print the support-vector fraction for the two kernels side by side, so that the diagnostic of slide 57 becomes two numbers you produced.

Slide 71 — The three families compared, on the same pumps and the same folds

Model	Accuracy, 5-fold cross-validation	Standard deviation over the 5 folds
Majority baseline	0.613	0.000
Logistic regression (lesson 4)	0.613	0.000
SVM, linear kernel	0.613	0.000
Gaussian Naive Bayes	0.933	0.010
k-NN, k = 5	0.944	0.006
SVM, RBF kernel	**0.947**	0.005
noise ceiling	*0.96*	

Everything in the lesson on the same 1,200 pumps, cross-validated the same way. The top three rows are all linear and all exactly at the base rate, with no spread at all: every fold calls every pump faulty. The bottom three are all near the ceiling, reached by three unrelated routes: remembering the neighbourhood, assuming independence, bending the space. The ceiling made exact: the generator flipped 49 labels, so a model that knew the true rule would score $1 - 49/1{,}200 = 0.959$ on these pumps.

The gap between the best and the worst is $0.947 - 0.613 = 0.334$, larger than any difference this course has shown between a default model and a carefully tuned one; lesson 5's hyperparameter search moved scores by a few hundredths. **Choosing the right family matters far more than tuning the wrong one**, and the way to know which family you need is to look at the data first.

Question — so the RBF SVM is the best model of the three?

Not on this evidence. Read the bottom three rows as comparable, not ranked. The gaps between them, 0.003 between k-NN and the RBF SVM, are smaller than one fold's standard deviation; $k = 5$ was chosen on these same folds, which flatters it slightly; and the RBF SVM got there with scikit-learn's defaults, $\gamma = 0.5$ and $C = 1$, untuned. Lesson 5's 200 splits showed how far a score moves when only the split changes. What the table does establish is the gap of 0.334 between the families.

Slide 72 — How to choose, in practice

Situation	Reach for
Few features, plenty of data, a curved boundary	k-NN, or an RBF SVM
Very many features, little data	Naive Bayes
A line separates the classes	Logistic regression or a linear SVM
You need probabilities you can trust	Check calibration on held-out data; logistic regression is the usual start; **not** Naive Bayes; an SVM gives only a score
The model must be small, or fast at prediction	Naive Bayes; an SVM with few support vectors; not k-NN
The signal is an interaction between features	k-NN or an RBF SVM, not Naive Bayes

Each row is a conclusion from today's evidence rather than general advice. k-NN and the RBF SVM follow the curved pump boundary. The three linear models tie at 0.613 only because the

pumps are not separable by a line; where a line does separate, it is the cheapest and most readable model there is. Naive Bayes was not calibrated (slide 45) and the SVM's +1.883 is a score, not a probability (slide 65); no model's probabilities are trustworthy by default, so the row says to check them on held-out data, as lesson 4 did. Naive Bayes is ten numbers (slide 32); an SVM is small only with few support vectors — the linear one kept 79% of the pumps (slide 57) — and k-NN keeps all 1,200. And on the interacting sensors k-NN and the RBF SVM scored 0.967 and 0.972 where Naive Bayes scored 0.404.

Question — "very many features, little data: Naive Bayes", after an hour of its failures?

It sounds wrong, and it is the row most worth a sentence. Naive Bayes estimates one small set of numbers per feature and per class, which is what very many features and little data require: a model that tries to learn how features act together needs far more data than a model that does not try. Text classification is the canonical case: tens of thousands of word-count features, an assumption that is transparently false, and a method that works anyway, because being roughly right in ten thousand dimensions beats being unable to estimate anything at all. Mind the version: for word counts it is `MultinomialNB`, which models counts; today's `GaussianNB` is for continuous readings.

Question — and when the decision has to be explained?

None of today's three does it well. A support vector machine in a lifted space offers no explanation the person affected would accept; k-NN can point at the neighbours, which is an example, not a reason. Lesson 3's Resources document covers why that is sometimes a legal requirement rather than a preference. Naive Bayes is also a good first baseline: it trains in one pass, so if a tuned model does not beat it you have learned something quickly and cheaply.

Slide 73 — What to take away

- **No line separates the pumps**: 0.613.
- **k-NN**: k sets bias against variance; in 100 random columns the nearest is 70% as far as the farthest.
- **Naive Bayes**: 0.933, but **0.404** on interactions.
- **SVM**: support vectors decide; kernels bend the line.
- **Choosing the family beats tuning**: 0.613 to 0.947.

The lesson's number, one more time: **0.613 to 0.947**. The same pumps, the same folds; a straight line stuck exactly at the base rate, a kernel reaching the noise ceiling. Choosing the family moved the score by a third; tuning never moved it that far.

The habit to keep is a pair: before reaching for a model, look at the shape of the data; after fitting one, check whether the assumption it depends on holds on your data, and check it properly. The within-class correlation of −0.006 said the assumption held; the squared distances said it did not; and the ellipse the model drew explained the 0.933. And one instinct to correct: more features is not free. It is nearly free for a linear model and expensive for anything built on a distance. Handout section 7 lists all of this with the section numbers.

Slide 74 — Homework

Exercise 6, discussed at the start of the lesson on Friday 27 November. Two stations: fit five models from the three families and the linear baseline, at both stations, and argue which one you would choose, and why. What matters in the exam is the reasoning in your notebook, not the accuracy: a well-argued choice that scores slightly lower beats a lucky winner with no justification. The argument must refer to the data, in the way the squared distances and the fitted ellipse explained Naive Bayes on the pumps. Cross-validate everything and report a spread, as lesson 5 set out, and keep the scaler inside the pipeline: at the exam it is one of the first things looked at.

Question — what does the last part of the exercise ask about the ceiling?

Two things can cap accuracy, and the exercise asks you to tell them apart: errors in the reference labels, and readings that overlap between the classes. Today's pumps had only the first: the rule was exact, 49 labels were flipped, and the ceiling was 0.959. A station whose readings overlap has a second limit, which no model can pass even with perfect labels. Work out which of the two you can compute at each station, and how close your best model came.